AI-POWERED SMART ORCHID CARE SYSTEM USING MULTI-MODAL MACHINE LEARNING

INDIVIDUAL THESIS

Vithanage P.V.P.M

(IT22062642)

Individual component: Smart Watering and Automated Fertilization for Vanda Orchids

Group ID: R26-SE-018

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) in Information Technology

Specialising in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Vithanage P.V.P.M

Student ID: IT22062642

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Hansi De Silva		
Co-supervisor: Ms. Thilini Jayalath		

ABSTRACT

Vanda orchids grow bare root, so the substrate moisture reading that automatic irrigation needs does not exist, and because they are watered daily, a grower's real decisions are when and how long to water each part of a shade house. Following design science research, this thesis presents Component 3 of the R26-SE-018 smart orchid care system, which plans and carries out watering, humidity tray filling and fertilization for each section of a *Vanda* shade house. ESP32 nodes measure air temperature, humidity and light per section, a master controller per house runs two pumps and confirms every pour, a cloud service plans each section at dawn and acts through the day, and the shared mobile application provides control, roles and full screen alarms. Random forest models trained on 58,436 section-days derived from eight years of ERA5 reanalysis for five Sri Lankan sites predict watering time and duration. On two withheld years the time error is 8.42 minutes (R^2 0.954) and on weather obtained after training 8.41 minutes, the noise floor of the expert rule labels. A classifier anticipates humidity falling below 60 % up to three hours ahead with an AUC of 0.986 on unseen years. It decides when a tray is filled, while the grower sets how much it pours. Feeds go only with water and never to dormant plants. Sensor number and placement are derived from a calibration period and scored on held-out time. Sections without a sensor are estimated by kriging. An end-to-end campaign of 19 scenarios found 25 defects, and all fixed now. Four nodes record in a farm house, where sections differed by 3.84 °C on average in the first hours. The author built every part of the component. The models reproduce an expert rule, and validating it against grower outcomes is the main future work.

Keywords: Vanda orchid, precision irrigation, vapour pressure deficit, random forest, sensor placement

ACKNOWLEDGEMENTS

I am grateful to my supervisor, Ms. Hansi De Silva, and co-supervisor, Ms. Thilini Jayalath, for their guidance throughout this research, and to the members of the progress review panels, whose questions on how sensor placement is decided, how readings vary and how the watering times are known to be right shaped much of the methodology.

I thank the members of the research group, Rathnamalala D.M.J.P, Nimla Samadinee L.D and Wickramasinghe D.P, for building the shared system with me.

I thank the owner and staff of the orchid farm for allowing the site visit and the installation of the sensor nodes, and for explaining how *Vanda* are watered and fed in practice, and the decision to redesign the component around the time and duration of watering came from those conversations.

No external funding was received. Use of generative AI tools in this work is disclosed in Appendix B.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 1

1.3 Problem statement 3

1.4 Research gap 3

1.5 Aim and objectives 3

1.6 Research questions or hypotheses 4

1.7 Scope limitations and assumptions 5

1.8 Individual contribution 5

1.9 Thesis structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review method and scope 7

2.2 Domain and theoretical foundations 7

2.3 Related research 9

2.4 Existing systems products and techniques 10

2.5 Gap synthesis and conceptual framework 10

3 RESEARCH METHODOLOGY 13

3.1 Research design 13

3.2 Data participants or study materials 15

3.3 Experimental or study procedure 18

3.4 Measures and evaluation metrics 18

3.5 Data analysis 19

3.6 Ethics privacy security and responsible AI 20

3.7 Reproducibility and research artefacts 20

4 PRODUCT AND SOLUTION DESIGN 22
4.1 Requirements 22
4.2 Component architecture within the complete solution 23
4.3 Design alternatives and decisions 24
4.4 Detailed design 25
4.5 Standards and professional practice 37
5 IMPLEMENTATION INTEGRATION AND TESTING 38
5.1 Development environment and technology stack 38
5.2 Component implementation 38
5.3 Integration with the other components 39
5.4 Verification and software testing 39
5.5 Deployment and operation 41
6 RESULTS AND VALIDATION 42
6.1 Research results 42
6.2 Product and technical results 45
6.3 User or stakeholder validation 47
6.4 Integration results 49
6.5 Summary of findings 49
7 DISCUSSION 51
7.1 Interpretation of findings 51
7.2 Comparison with literature baselines and existing solutions 52
7.3 Research contribution 52
7.4 Product contribution 52
7.5 Limitations and threats to validity 52
7.6 Implications 53
8 COMMERCIALISATION AND SUSTAINABILITY 54

8.1 Value proposition and target users 54
8.2 Market and competitive position 54
8.3 Business or sustainability model 55
8.4 Scalability intellectual property and compliance 55
8.5 Contribution to the Sustainable Development Goals 56
9 CONCLUSION AND FUTURE DIRECTIONS 57
9.1 Conclusion 57
9.2 Achievement of objectives 57
9.3 Individual research and product contributions 58
9.4 Future directions 58
REFERENCES 59
GLOSSARY 63
APPENDICES 65
Appendix A Research instruments or extended results 65
Appendix B Academic integrity and AI use disclosure 71

LIST OF FIGURES

Figure 1.1: Shade houses of the orchid farm visited on 17 June 2026: (a) and (b) benches of *Vanda* and related orchids under shade netting, (c) a house with a block wall on one side 2
Figure 2.1: Conceptual framework of the component 12
Figure 3.1: Distribution of the generated watering-time and duration targets over 58,436 section-days 17
Figure 4.1: Architecture of Component 3 within the shared solution 24
Figure 4.2: Wiring of a sensor node, generated from the firmware's pin map 26
Figure 4.3: What the automation engine does over a day 31
Figure 4.4: The eight steps of the placement analysis 33
Figure 4.5: Calibration in the application: (a) the even spread proposed for a new house, (b) linking a node to a section, (c) the chosen placement with one node's reach drawn on the plan (screens (b) and (c) from the rehearsal with virtual nodes and injected biases; Original in Color) 35
Figure 4.6: The application: (a) sign-in, (b) the farm with its houses, (c) sections with readings and today's plan, (d) choosing a house's pump channels (test-bench house; Original in Color)..... 36
Figure 4.7: Water Now: (a) choosing sections, (b) queued at the master, (c) the master running the pour, (d) finished and confirmed by the master (test bench with a virtual master; Original in Color) 36
Figure 4.8: An alarm on a locked phone (a), the action it leads to (b) and its confirmation (c) (test-bench house; Original in Color)..... 37
Figure 6.1: Mean absolute error of the watering models under the three designs, against the noise built into the labels 42
Figure 6.2: Feature importance in the deployed watering-time and duration models 43
Figure 6.3: Out-of-time performance of the humidity-drop classifier: ROC curve and threshold sweep 44
Figure 6.4: Temperature, relative humidity and VPD recorded by the four nodes of house H1 on 1 Oct 2026 (grey: set-up time, excluded; Original in Color)..... 48

Figure A.1: Master controller: relay board and the two pumps 70
Figure A.2: The first made carrier board, checked on 9 October 2026 71

LIST OF TABLES

Table 1.1: Components of the group project R26-SE-018 1
Table 1.2: The author's work on Component 3 6
Table 2.1: Comparison of the reviewed systems with this work 11
Table 3.1: Main iterations of the second design 14
Table 3.2: Outdoor conditions at the six sites, ERA5 2017 to 2024 15
Table 3.3: Section microclimates used to expand each site's weather 16
Table 3.4: Measures used in the evaluation 19
Table 3.5: Scripts that produce the reported results 21
Table 4.1: Requirements of the component 23
Table 4.2: Main design decisions 25
Table 4.3: Pin assignments of the sensor node and master controller 26
Table 4.4: Deployed estimators 30
Table 5.1: Technologies used by the component 38
Table 5.2: Size of the component's code 39
Table 6.1: Error of the watering models under three evaluation designs 42
Table 6.2: Precision, recall and F1 of the humidity-drop classifier at four thresholds (2023 to 2024) 44
Table 6.3: Measured response of the deployed system in the end-to-end campaign 46
Table 6.4: Defects found by the end-to-end campaign, by severity 47
Table 6.5: Readings per section in house H1, 1 Oct 2026 10:30 to 13:38 49
Table 7.1: Limitations of the work 53
Table 8.1: Cost basis of the hardware 55
Table 9.1: Achievement of the objectives 57
Table A.1: End-to-end scenarios and their results 66
Table A.2: Layout of the Realtime Database used by Component 3 69
Table A.3: REST endpoints of Component 3 70
Table B.1: Use of generative AI and automated tools 71

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AUC	Area under the ROC curve
ECMWF	European Centre for Medium-Range Weather Forecasts
ERA5	Fifth-generation ECMWF atmospheric reanalysis
FAO	Food and Agriculture Organization of the United Nations
FCM	Firebase Cloud Messaging
GPIO	General-purpose input/output
HTTPS	Hypertext Transfer Protocol Secure
IoT	Internet of Things
I²C	Inter-Integrated Circuit (bus)
LKR	Sri Lankan rupee
MAE	Mean absolute error
NPK	Nitrogen, phosphorus and potassium
REST	Representational state transfer
RH	Relative humidity
ROC	Receiver operating characteristic
R^2	Coefficient of determination
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
SSPOR	Sparse sensor placement optimization for reconstruction
TLS	Transport Layer Security
VPD	Vapour pressure deficit

Abbreviation	Meaning
VPS	Virtual private server

1 INTRODUCTION

1.1 Overall project context

This thesis is about one component of the research project R26-SE-018, *AI-Powered Smart Orchid Care System Using Multi-Modal Machine Learning*, which four students at the Sri Lanka Institute of Information Technology (SLIIT) built together. The four components share one mobile application, one backend service and one cloud database (Table 1.1). Three of them analyse images and records of the plants. Mine, Component 3, is the one that moves water.

Table 1.1: Components of the group project R26-SE-018

No.	Component	Member
1	Hybrid pollination and compatibility analysis	IT22065230, Wickramasinghe D.P
2	Orchid disease detection and treatment recommendation	IT22064936, Rathnamalala D.M.J.P
3	**Smart watering and automated fertilization**	**IT22062642, Vithanage P.V.P.M**
4	Orchid growth stage recognition and bloom prediction	IT22083500, Nimla Samadinee L.D

1.2 Individual component context

Vanda orchids are grown commercially for cut flowers and potted plants. They have no potting medium. Their thick aerial roots hang in the air, in open baskets or on bare benches, and take up water through a spongy outer layer called the velamen. In Sri Lanka they are grown in shade houses roofed and walled with shade netting (Figure 1.1). Every day the grower has to decide when each part of a house should be watered and for how long, keep the air around the plants humid enough, and feed the plants without harming the roots.



(a) (b) (c)

Figure 1.1: Shade houses of the orchid farm visited on 17 June 2026: (a) and (b) benches of *Vanda* and related orchids under shade netting, (c) a house with a block wall on one side

Source: photographs taken during the site visit

Three things about these houses shaped the work. A house is not one climate. The edge that gets the afternoon sun is hotter and drier than the shaded back corner, by several degrees on the farm where the system now runs (Section 6.3). The plants have no substrate, so the soil moisture reading that automatic irrigation normally depends on does not exist. And the houses are built and looked after by hand by growers who are not technology specialists, so any equipment has to be cheap, robust and easy to replace.

I chose this topic after seeing how watering was done on the farm I visited. The watering was manual, and I wanted to see whether it could be upgraded. I also wanted to learn how Internet of Things (IoT) devices are built and to work with them myself, which I had not done before.

I talked the idea over with the grower on that farm. The grower said that watering the whole farm every day is hard work. If a house has several sections, and there were options to control the whole farm at once, the grower would like that, and a history of the waterings would be useful as well. These conversations were informal and I did not run a formal study, but they match what the system offers. It has a farm wide automatic care switch, waters several sections in one run and keeps a record of every watering.

Component 3 is a complete system. It has low cost sensor nodes and a pump controller in the shade house, a cloud service that plans and carries out watering, humidity tray filling and feeding for every section, and the screens of the shared mobile application through which the grower supervises it.

1.3 Problem statement

> *How can a Vanda shade house, which offers no substrate in which to measure water, be watered at the right time and for the right duration in each of its microclimates, with the humidity around the plants kept within their tolerated band and fertilizer delivered only with water, by a system whose number of sensors is justified by measurement rather than assumed, and which a non-specialist grower can trust to run unattended?*

The question has a research side and a product side. The research side asks whether watering can be scheduled from air conditions alone, whether humidity can be kept in band by acting early, and how many sensors a house needs. The product side asks whether such a system can be built from commodity parts and run safely on a real farm, where networks drop, sensors fail and the grower must always be able to tell what actually happened.

1.4 Research gap

The systems I reviewed in Chapter 2 decide watering from the moisture of a soil or substrate, which *Vanda* do not have. They answer whether to water, not when and for how long. They handle humidity separately or not at all, they treat fertilization as advice, and none of them works out how many sensors a house needs. Their evaluations are short, at single sites, and where machine learning is used, they report results on random splits of time series data. Section 2.5 sets the gap out in detail.

1.5 Aim and objectives

Aim. To design, build and evaluate a sensor driven system that plans and carries out watering, humidity tray filling and fertilization for each section of a *Vanda* shade house, using machine learning models validated on unseen years and a sensor layout derived from the house's own measurements.

Objectives.

(1) To build a low cost sensing node and a house level pump controller that report and act over Wi-Fi, keep their readings when the network fails, and confirm every pour they carry out.

(2) To train models that predict, for each section and day, the watering time within the 06:00 to 09:00 window and the watering duration, and to validate them on years excluded from training.

(3) To build a humidity tray loop in which the models decide when a tray is filled, including up to three hours before humidity leaves the band, and the grower decides how much water a fill pours.

(4) To schedule fertilization by growth stage, at reduced strength, delivered only with a watering and never to a dormant plant.

(5) To derive the number and placement of sensors in a house from a calibration period of real readings, measure how far one node's reading remains representative, and estimate sections without a sensor by kriging.

(6) To deliver the system through the mobile application with role based accounts, alarms that reach a locked phone and safety rules that no model can override, and to verify it end to end on the deployed service and on the farm.

1.6 Research questions or hypotheses

(1) **RQ1.** Can the time and duration of a *Vanda* watering be predicted from air temperature, humidity and light alone, and does the prediction hold on years the model has never seen?

(2) **RQ2.** Can the humidity around the plants be kept in the 60 to 80 % band by acting before it leaves the band, using only the section's own recent readings?

(3) **RQ3.** How many sensors does a given house need, how far does one sensor's reading stay representative, and how well can a section without a sensor be estimated from its neighbours?

(4) **RQ4.** What must the hardware, firmware, cloud service and application do so that every automated action is safe, confirmed by the hardware and visible to the grower, including when the network, a sensor or the controller fails?

1.7 Scope limitations and assumptions

The scope is the sensing and pump control hardware of one or more shade houses, its firmware, the cloud service with its models, rules and analyses, and the grower facing screens of the mobile application for watering, humidity, feeding, calibration, alarms and accounts.

Images play no part in this component's decisions, because camera based analysis belongs to Components 1 and 2.

I assume that the watering targets can come from an expert rule (Section 3.2.2), that the farm has Wi-Fi coverage (a 4G router on the present farm), and that the DHT22 sensor meets its stated accuracy of ±0.5 °C and ±2 % relative humidity (RH), which the placement analysis uses as its tolerance.

1.8 Individual contribution

I designed, built and evaluated every part of Component 3 described in this thesis (Table 1.2). The account system I built for the component also provides sign in for the whole application, and the backend server I set up for it hosts the services of all four components.

Table 1.2: The author's work on Component 3

Area	Work carried out
Problem and design	Requirements from the farm visit and the grower, the first design and its abandonment, and the second design around watering time, duration, humidity trays and measured sensor placement
Hardware	Sensor node and master controller, pin map, wiring guide generated from the firmware, carrier board design pack and the check of the first made board, and assembly of the four farm nodes
Firmware	One firmware with normal and quiet builds, covering readings, store-and-forward, heartbeat and ping, Wi-Fi setup hotspot, self-registration, the master command queue with acknowledgements and safety refusals, and the offline fallback
Data and models	Eight years of reanalysis weather for six sites, label rules, training of the seven estimators, and random, out-of-time and unseen weather validation
Cloud service	The automation engine, plans, humidity tray loop, second watering rule, fertilization schedule and guards, device health checks, alarms, kriging of sections without a sensor, the eight step placement analysis, and deployment on the server
Mobile application	The farm, section, Today, Run, Alarm, calibration, placement result and team screens, and the native Android alarm service
Testing and deployment	Unit and regression tests, the digital twin, the node bench, the rehearsal harness, the end-to-end campaign and the field deployment of house H1

1.9 Thesis structure

Chapter 2 reviews the literature and existing practice and sets out the gap. Chapter 3 describes the research design, data, procedures and metrics. Chapter 4 describes the product and the decisions behind it, and Chapter 5 its implementation, integration and testing. Chapter 6 reports the results and validation, Chapter 7 discusses them with the limitations of the work, Chapter 8 covers commercialisation and sustainability, and Chapter 9 concludes.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The review covers four areas, namely the water relations of epiphytic orchids, automatic irrigation built on sensors and the Internet of Things (IoT), smart farming systems for orchids, and the methods this component depends on (random forests, time series evaluation, weather reanalysis, spatial estimation and sensor placement). Sources were found through IEEE Xplore, the SLIIT research repository and Google Scholar, with searches combining orchid, *Vanda*, irrigation, humidity, IoT and machine learning, and by following the references of the papers found. Peer reviewed papers and authoritative technical or horticultural sources were preferred.

Every reference in this thesis was checked against the publisher's record, the institutional repository or the citing literature for its authors, title, venue and year.

2.2 Domain and theoretical foundations

2.2.1 Orchid cultivation and the Vanda grower

Commercial orchid growing in Sri Lanka suits the warm, humid lowlands. A chapter on Sri Lankan cut flower production in a regional study by the Food and Agriculture Organization of the United Nations (FAO) describes the climate of the Western Province as favourable for *Dendrobium*, *Vanda* and *Phalaenopsis*, and reports that almost all orchid cultivation took place under shade netting, with most growers using mist irrigation [1]. The floriculture sector remains one of small and medium sized producers, many of them producer-exporters operating their own farms [2].

2.2.2 How a Vanda takes up water

Vanda are epiphytes, and their roots are covered by the *velamen radicum*, a sheath of dead cells several layers deep. Zotz and Winkler [3] showed that the velamen of many orchid species takes up liquid water within seconds, while water is lost again by evaporation over a much longer period, and that it plays a measurable part in capturing nutrients from the first water that reaches it. The physiology of tropical orchids in commercial production is reviewed by Hew and Yong [4].

Two consequences follow for automation. There is no medium in which to place a moisture probe, and a probe pressed against a living root reports on one root of one

plant. And the useful decision is not *whether* to water, because practical guidance for *Vanda* is to give generous water, as often as daily when air circulation is good, in the morning, and less in the cooler months [5]. The grower consulted for this project confirmed that daily watering is standard practice. What remains open each day is the *time* and the *amount*. Water given too late in a hot morning reaches a root that has been dry for hours, and water given in excess in humid air leaves roots wet into the night, which favours rot.

2.2.3 Evaporative demand and the vapour pressure deficit

How fast an exposed root dries depends on how strongly the air draws water from it. The quantity that captures this is the vapour pressure deficit (VPD), the difference between the vapour pressure the air could hold at its temperature and the vapour pressure it holds. The FAO guidelines for crop evapotranspiration give the standard expression for the saturation vapour pressure [6], used in this work (Section 4.4.4). VPD is a primary driver of plant water loss and stomatal behaviour [7]. For a *Vanda*, whose roots are in direct contact with the air, it is close to a direct measure of the drying power the plant experiences.

2.2.4 Fertilizing Vanda

Vanda are heavy feeders compared with most orchids. Horticultural guidance recommends a balanced fertilizer at one quarter to one half of label strength, applied with the water and reduced in the cooler months, with occasional high phosphorus feeding to support flowering [5]. Two principles are common to this guidance and to the grower's practice. Feed at reduced strength, and never feed onto dry roots, because a concentrated salt solution damages a dry velamen. Fertilization is therefore a property of a watering rather than a separate task.

2.2.5 Random forests and the evaluation of time series models

Random forests [8] are ensembles of decision trees trained on bootstrap samples with random feature selection at each split. They model non-linear interactions between inputs with little tuning and expose a feature importance measure, and the implementation used here is scikit-learn [9]. Two cautions shape their evaluation in this work. When observations close in time are strongly correlated, a random split lets a model be tested on the remainder of a day it has partly seen, and blocked or out of

time evaluation is recommended instead [10], [11]. And impurity based importances are biased towards continuous and correlated predictors [12].

2.2.6 Weather reanalysis

A model that schedules watering from the weather needs years of weather to learn from, and the farm visited for this project had no multi-year record of conditions inside its houses. ERA5 [13], the fifth-generation reanalysis of the European Centre for Medium-Range Weather Forecasts (ECMWF), reconstructs hourly near surface temperature, humidity and solar radiation on a global grid by assimilating historical observations from weather stations, satellites, balloons and aircraft into a weather model. It is real past weather, not generated data, on a grid of roughly 31 km. The Open-Meteo historical interface [14] serves ERA5 for any coordinate.

2.2.7 Spatial estimation and sensor placement

Kriging, introduced by Matheron [15] and developed by Cressie [16], is the best linear unbiased predictor of a quantity at an unmeasured location under a model of how similarity decays with distance, and it reports the variance of each estimate. This work uses the implementation of PyKrige [17]. Choosing sensor locations is itself a studied problem. Krause et al. [18] place sensors to maximize the information they give about unobserved locations of a Gaussian process, and Manohar et al. [19] choose them from historical snapshots of a field by QR pivoting on a learned low rank basis, an approach called sparse sensor placement optimization for reconstruction (SSPOR) and implemented in the PySensors package [20].

2.3 Related research

Orchid irrigation and climate control. Maleesha and Silva [21] built an automated water distribution system for the *Dendrobium Phalaenopsis* group, monitored through a cloud dashboard. It waters when soil moisture falls to 30 %, reports water use efficiency between 60 % and 95 %, and proposes bringing temperature, humidity and forecasts into the control as future work. Fauzy et al. [22] monitored the humidity of an orchid room with an ESP8266 and a DHT-11, predicted the next humidity value and switched a mist maker on when the prediction fell below 50 %. The system predicts the environment, not the plants' need, and does not water. Drouaillet-Mejía et

al. [23] installed an open source automated irrigation system in a greenhouse growing *Phalaenopsis*, *Dendrobium* and *Vanda* in Mexico. Regulating water from substrate moisture, temperature and humidity, it reduced water use from 90 L (manual) to 5 L (automated) over September to December 2023, although the source does not state the period each figure covers and reached 100 % plant survival against 13 to 20 % mortality under manual care, in a before and after comparison at one site.

IoT irrigation in general. García et al. [24] review the sensors and IoT architectures used for irrigation, and soil moisture remains the central measurement and the control is most often a threshold on it. Tephila et al. [25] describe a representative ESP8266 system with soil moisture, temperature, humidity and water level sensors. Goap et al. [26] combine recent readings with weather forecasts to predict soil moisture for the coming days using machine learning on open source hardware and software. All rest on a substrate in which moisture can be measured.

2.4 Existing systems products and techniques

In practice, orchid houses are watered by hand or by mist and drip lines on manual valves or fixed timers [1], and commercial irrigation controllers, as surveyed by García et al. [24], add a soil moisture threshold to the timer. OrchiZen [27], an integrated smart farming platform for *Dendrobium*, *Vanda* and *Phalaenopsis*, combines machine learning and IoT sensing in modules for irrigation advice, species recommendation, UV based disease treatment, light optimization and fertilizer guidance. As described in its published abstract, its irrigation and fertilizer functions are advisory. The techniques this component builds on, random forests, ordinary kriging and data driven sparse sensor placement, are available as mature open source libraries [9], [17], [20], but none of the reviewed agricultural systems applies the last two to deciding how many sensors a house needs.

2.5 Gap synthesis and conceptual framework

Table 2.1 compares the reviewed systems against the capabilities this component sets out to provide. Where a capability is not described in a source, it is marked as not reported rather than absent.

Table 2.1: Comparison of the reviewed systems with this work

Work	Crop	Water decision based on	Predicts time and duration	Humidity loop	Fertilization	Sensor number measured
Maleesha and Silva [21]	Dendrobium Phalaenopsis	soil moisture threshold (30 %)	No	No	No	No
Fauzy et al. [22]	orchid room	predicted room humidity (misting)	No	Yes, misting	No	No
OrchiZen [27]	Dendrobium, Vanda, Phalaenopsis	environmental sensing, advisory	Not reported	Not reported	Advice	No
Drouaillet-Mejía et al. [23]	Phalaenopsis, Dendrobium, Vanda	substrate moisture, T, RH	No	No	No	No
Tephila et al. [25]	field crops	soil moisture and other sensors	No	No	No	No
Goap et al. [26]	field crops	predicted soil moisture	Partly (when)	No	No	No
This work	**Vanda shade houses**	**air temperature, humidity, VPD**	**Yes, per section**	**Yes, anticipatory**	**Scheduled, with water**	**Yes, from calibration**

(1) **No measurement to control on.** Every reviewed watering controller decides from the moisture of a soil or substrate, which *Vanda* do not have.

(2) **The wrong decision is automated.** Where watering is daily practice, a controller that answers *whether* to water answers a question the grower does not have, and none predicts time and duration from the conditions the plants experience.

(3) **Humidity is handled separately, if at all.** For an epiphyte that takes water from the air, watering and humidity interact and need to be managed together, with humidity raised without wetting the roots.

(4) **Fertilization is advice, not part of the watering.** None of the systems schedules a feed into the water the plants are about to receive or refuses to feed a dormant plant.

(5) **The number of sensors is assumed, not measured.** No reviewed system derives how many sensors a house needs, estimates the parts without one, or tests the layout on data it did not see.

Figure 2.1 summarizes the conceptual framework that answers these gaps. Air measurements from each section, a calibration period that decides where sensors stay, and the grower's settings feed a set of decisions (when and how long to water, when to fill the humidity tray, whether to feed), which are bounded by safety rules no model can override, carried out by a controller that confirms what it did, and reported to the grower, whose actions and the confirmed outcomes are recorded for later retraining.

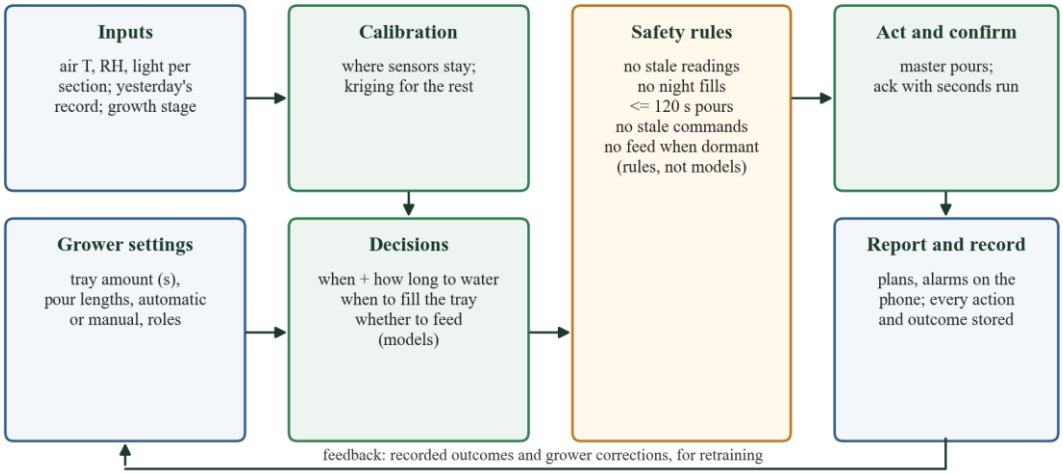


Figure 2.1: Conceptual framework of the component

3 RESEARCH METHODOLOGY

3.1 Research design

The work follows design science research [28], in which knowledge is produced by building an artefact that addresses a real problem and evaluating it rigorously, and the process model of Peffers et al. [29], whose steps are to identify the problem, define the objectives of a solution, design and develop, demonstrate, evaluate and communicate. Evaluation combines quantitative model validation on withheld data, tests of the analysis method on data with a known answer, an end-to-end test campaign on the deployed system, and a field deployment.

The process was iterative. The **first design** (April to June 2026) predicted, from a capacitive probe pressed against the roots of one plant and the air conditions, whether the plant needed water. Trained on a synthetic dataset of 1,000 rows it reached an F1 score of 0.746 against a threshold rule's 0.017 on the same labels, but two problems no score could fix ended it. A probe cannot practically be fitted to every plant, and the grower confirmed that *Vanda* are watered daily anyway, so *whether* was the wrong question. The **second design** (from July 2026), which this thesis describes, reformulates the problem around the decisions the grower actually makes and replaces the per plant probe with air measurements. Table 3.1 lists its main iterations.

Table 3.1: Main iterations of the second design

Period	Change	Trigger
Jul 2026	Watering time and duration models, and two control loops per section (watering, humidity tray)	Daily watering is mandatory, so the useful decision is when and how long
Jul 2026	Training data changed from a synthetic climate to ERA5 reanalysis	The synthetic climate saturated the stress scale every day, so the model always answered 06:00
Aug 2026	Digital twin of a farm writing to the real database	Automatic mode diagnosed empty trays but never filled them, and three further defects
Aug 2026	One firmware for all nodes, two pumps per house, command acknowledgement and a master command queue	The application's Water Now reached a document no physical node read
Sep 2026	Multi-tenant accounts with admin, operator and viewer roles	A farm is sold with an admin account that manages its own users
Oct 2026	Calibration and placement analysis, and kriging of sections without a sensor	Progress review panel asked to see what data decides placement, and one node's coverage
Oct 2026	Store and forward, quiet recording nodes, device health checks	A pump that did not run at an earlier demonstration was diagnosed only afterwards
Oct 2026	End-to-end campaign, with 25 defects found and all fixed	Full run of every scenario on the deployed service before field deployment
10 Oct 2026	Tray level probe removed, so the grower sets the tray fill amount and the models decide when	The probe read 0 % for thirty hours on the farm, and an unplugged or floating probe reads as a plausible empty or full tray, so it needed fault handling of its own, and the grower already knows the tray's volume
11 Oct 2026	Offline fallback, so that a master waters from its own copy of the day's plan when the cloud is gone	A missed day of watering whenever the engine or the farm's internet is down

At the PP2 review on September 2026 the panel asked for six things. They were the data that decides where the sensors go, shown as a flow, how the variation in sensor readings is handled, the evaluation metrics, the maximum coverage of one node, how the watering times were verified, and that the application should be able to tell when an IoT part, such as a sensor, is disabled. The answers are the eight step placement analysis with its cleaning, bias removal, variation and test period error steps (Section 4.4.8), the measure of one node's reach, the out of time validation of the watering

models (Section 6.1.1), and the device health checks that raise an alarm (Section 4.4.11).

3.2 Data participants or study materials

3.2.1 Training data from ERA5 weather at six sites

The models are trained on hourly ERA5 reanalysis [13], retrieved through the Open-Meteo archive interface [14] for six locations spanning Sri Lanka's climate zones, namely Peradeniya (hill country, wet zone), Battaramulla (coastal wet zone), Galle (southern coast), Anuradhapura (dry zone), Jaffna (northern dry zone) and Nuwara Eliya (high altitude). For each, the hourly 2 m temperature, 2 m relative humidity and shortwave radiation were downloaded for 2017 to 2024, giving 70,128 hours per site and 420,768 hours in all (Table 3.2). No generated weather and none of the project's resampled weather files are used for training or reported as results.

Table 3.2: Outdoor conditions at the six sites, ERA5 2017 to 2024

Site	Hours	Mean T (°C)	Min T (°C)	Max T (°C)	Mean RH (%)	Used
Anuradhapura	70,128	26.7	18.2	37.5	80.0	Yes
Battaramulla	70,128	26.6	18.3	35.5	84.0	Yes
Galle	70,128	26.4	19.8	35.2	84.6	Yes
Jaffna	70,128	27.8	19.9	37.1	79.4	Yes
Peradeniya	70,128	24.1	14.4	35.9	81.2	Yes
Nuwara Eliya	70,128	15.9	4.8	25.0	86.1	No

Nuwara Eliya was excluded because *Vanda* are not farmed at its altitude, and its minimum of 4.8 °C stretched the percentile scale of the stress score (Section 3.2.2) so far that an ordinary lowland day scored as extreme. The remaining five sites provide 350,640 hours.

Reanalysis describes the open air. Inside a shade house, shade cloth cuts the light, the structure warms in the sun, and transpiring plants and reduced air movement raise the

humidity. The conversion applied to every hour is given in (3.1) to (3.3), where R is the outdoor shortwave radiation in W/m² and $s = \min(R/1000, 1)$ the fraction of full sun, so the warming and the reduction in the humidity lift are greatest at midday and vanish at night.

$$T_{in} = T_{out} + 3.5\, s \tag{3.1}$$

$$RH_{in} = RH_{out} + 7\,(1 - 0.45\, s) \tag{3.2}$$

$$L_{in} = 0.45 \times 120 \times R \tag{3.3}$$

Each hour of each site is expanded into four section microclimates that differ in exposure (Table 3.3), with offsets applied in proportion to the sun, giving 1,402,560 section-hours. The watering models are trained on daily rows, one per site, section and day that has a dawn reading (05:00) and a previous day, which gives 58,436 rows. One flaw was found while preparing this thesis. The first day of each site after the first takes its "yesterday" from the last day of the previous site, which affects 16 of the 58,436 rows.

Table 3.3: Section microclimates used to expand each site's weather

Section type	Light exposure	T offset at full sun (°C)	RH offset at full sun (%)
bright-south	1.00	+1.8	−8
open-west	0.75	+1.0	−4
shaded-east	0.45	−0.8	+4
back-corner	0.35	−1.2	+6

Source: ml_pipeline/train_on_real_data.py

3.2.2 Targets from an expert watering rule

No dataset records when a skilled grower would have watered each section on each of these days, so the targets are generated by an expert rule applied to the real weather. After a day of high evaporative demand, roots start the morning drier and should be watered earlier and for longer. Yesterday's demand is summarised as a stress score S ((3.4)), the mean of three terms placing yesterday's peak temperature, mean humidity and mean VPD within the 5th to 95th percentile range of all section-days, with a =

$(T_{peak} - T_5) / (T_{95} - T_5)$, $b = (RH_{95} - RH_{mean}) / (RH_{95} - RH_5)$ and $c = (VPD_{mean} - VPD_5) / (VPD_{95} - VPD_5)$. For the deployed model these ranges are 28.4 to 37.0 °C, 76.2 to 95.8 % and 0.15 to 1.13 kPa.

$$S = \text{clip}((a + b + c) / 3, 0, 1) \tag{3.4}$$

The target time in hours and duration in seconds are given by (3.5) and (3.6), where e is the section's light exposure and the noise terms are drawn as $\varepsilon_h \sim N(0, 0.18^2)$ and $\varepsilon_d \sim N(0, 3^2)$. They stand for the day to day variation of a real grower and set a floor under any error. A predictor that knew the rule exactly would still score about 8.62 minutes on the time and 2.39 seconds on the duration (Figure 3.1).

$$\text{hour} = \text{clip}(9 - 3S + \varepsilon_h, 6, 9) \tag{3.5}$$

$$\text{duration} = \text{clip}(45 + 55S + 14 (e - 0.6) + \varepsilon_d, 30, 120) \tag{3.6}$$

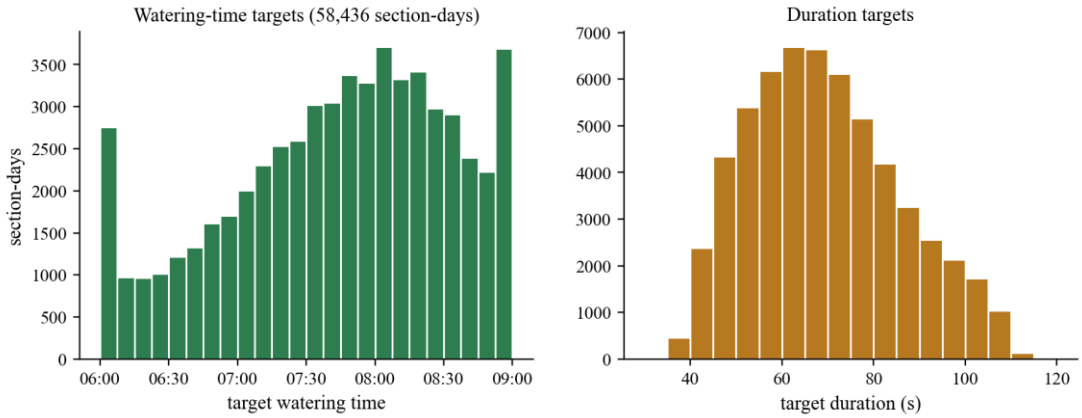


Figure 3.1: Distribution of the generated watering-time and duration targets over 58,436 section-days

Source: ml_pipeline/results/watering_real.csv

> *Because the labels come from a rule, a model trained on them can at best learn the rule. Good scores show that the model reproduces the rule on weather it has not seen, but they do not show that the rule is the best watering for the plants, which requires grower decisions and plant outcomes recorded over at least a season.*

The humidity tray anticipation target, by contrast, is not produced by a rule. It is whether the converted humidity series actually fell below 60 % within the next three hours.

3.2.3 Field data and participants

The field study uses one shade net house on the farm, registered as house H1 "Test" (6m by 3m, four sections), with one node per section recording from 1 October 2026 (Section 5.5.1). The grower who owns the farm took part by giving access, describing practice, and making operational choices such as skipping co-location. No personal data is collected about anyone, and no formal user study was conducted.

3.3 Experimental or study procedure

Model training and evaluation. Each model is trained with fixed random seeds on standardised inputs. The watering models are evaluated in three designs, all reported. The first is a random 80/20 split of 2017 to 2024, for comparison only. The second is an out-of-time split trained on 2017 to 2022 and tested on 2023 and 2024. The third is an unseen weather test of the model trained on all eight years against Peradeniya weather from 1 January 2025 to 15 August 2026 (2,364 section-days), downloaded after training. The tray anticipation model is evaluated out of time on the 47,515 daylight hours of 2023 and 2024. The validation script writes its results to a file, and every figure quoted in this thesis is read from that file.

Placement calibration. When a house is set up, one node records in each section for a calibration period, and the analysis opens after at least one day, at least 400 readings per section and a latest reading at most two hours old, and follows the eight steps of Section 4.4.8. Before use on a farm, the method was run on data with a known answer, namely injected sensor biases and nodes on a known temperature gradient.

End-to-end campaign. On October 2026 every scenario of the component was run on the deployed backend and the real application on the grower's phone, against a simulated test house whose four virtual quiet boards and virtual master follow the firmware's protocol, so that faults could be produced on demand without risking a real pump.

Field deployment. On October 2026 four nodes were installed in house H1.

3.4 Measures and evaluation metrics

Table 3.4 lists the measures used to answer each question, and why each was chosen.

Table 3.4: Measures used in the evaluation

Question	Measure	Why this measure
Watering time and duration accuracy	Mean absolute error (MAE) in minutes and seconds, and coefficient of determination (R^2)	In the units the grower uses, and compared with the label noise floor
Humidity drop anticipation	Area under the receiver operating characteristic (ROC) curve (AUC), and precision, recall and F1 at four thresholds	Separates ranking skill from the choice of operating point
Sensor bias recovery	Error of the recovered offset (°C, % RH)	Against a known injected bias
One node's reach	Coverage radius (m) where the fitted difference reaches sensor accuracy	Directly answers how far one node represents
Estimating a section without a sensor	Leave one out kriging error and test period error per placement method	Error on readings the method never saw
Fault detection	Minutes from fault to alarm	Measured on the deployed system
System correctness	Defects found and fixed, and automated tests passing	Each defect has a regression test
Variation on the farm	Spread between sections against two sensor errors	Whether a house can be treated as one climate

3.5 Data analysis

Model errors are computed on held-out data only. Feature importance is read from the fitted forests' impurity based importances and interpreted against the structure of the label rule. The anticipation classifier's operating threshold is chosen from a sweep of four thresholds by weighing the cost of each kind of error (Section 4.4.4). The placement analysis is computed by the backend from the stored readings, in 10 minute periods, with every reported error taken from the test part of a time ordered split (Section 4.4.8). The farm readings are summarised per section and as the spread

between the warmest and coolest section in each 10 minute period in which all four nodes reported.

3.6 Ethics privacy security and responsible AI

The component collects measurements of air, light and equipment state, and the actions taken. It collects no personal data beyond the account details of the farm's own users. Each farm's data is held under its own tenant in the database and served only to that farm's signed-in users, whose role (admin, operator or viewer) the server checks on every action. Traffic is encrypted with Transport Layer Security (TLS).

Responsible use of the models is built into the design. Every decision a model makes is bounded by rules it cannot override (no watering from readings more than two hours old, no tray fill at night, no pour over 120 seconds or from a command older than 15 minutes, no feed to a dormant plant). The grower can switch automation off for the farm or a section and still receive the same decisions as alarms. Every automatic action is recorded with the conditions that led to it and the seconds the hardware actually ran, and the limits of the models, rule based labels above all, are stated in the application as well as in this thesis. The use of generative AI tools in the work is disclosed in Appendix B.

3.7 Reproducibility and research artefacts

All code, firmware and tests are kept in the group's Git repository, and the trained models, which are too large for the repository, are kept with the training scripts. Table 3.5 lists the scripts that produce each reported result, and every number quoted in Chapter 6 is read from the files these scripts write.

Table 3.5: Scripts that produce the reported results

Result	Script or file
Weather for six sites, 2017 to 2024	ml_pipeline/fetch_multi_site.py
Watering models and their labels	ml_pipeline/train_on_real_data.py (labels in watering_real.csv)
Random, out-of-time and unseen weather validation	ml_pipeline/validate_out_of_time.py → backend/app/data/watering_validation.json
Tray dose and anticipation models	ml_pipeline/train_tray_v3.py → tray_v3.pkl, tray_v3_metrics.json
Day ahead forecast	ml_pipeline/build_forecast_model.py, improve_forecast_with_weather.py
Placement analysis	backend/app/services/placement_analysis.py and its tests
Backend tests	backend/tests (pytest)
Figures and tables of this thesis	Thesis/src/make_figures.py, then Thesis/src/build_template.py

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements in Table 4.1 were drawn from the farm visit and the grower's practice (Section 1.2), from the gaps in Section 2.5, and from the review panels, and were refined as testing exposed failure modes. Each is traced to where it is met.

Table 4.1: Requirements of the component

ID	Requirement	Met by
FR1	Measure air temperature, humidity and light in every section and keep readings through network outages	Sensor node, store-and-forward
FR2	Plan each section's daily watering time and duration from its conditions	Watering models, engine
FR3	Fill each section's humidity tray when humidity is low or about to fall, pouring the amount the grower set	Tray loop
FR4	Schedule feeds by growth stage, only with a watering, never when dormant	Fertilization schedule
FR5	Derive how many sensors a house needs and where, and estimate sections without one	Calibration, placement analysis, kriging
FR6	Let the grower water and fill trays by hand, stop a pour, and set how long pours run	Mobile application
FR7	Alert a person when an action is needed or a device fails, even on a locked phone	Alarms, device health
NFR 1	Never act on stale or impossible readings, and never water at night or pour longer than 120 s	Safety rules in backend and firmware
NFR 2	Count an action as done only when the hardware confirms it	Master acknowledgements
NFR 3	Usable by non specialist growers	Simple Today screen, plain words
NFR 4	Only the farm's own users may see or change the farm, by role	Accounts, roles, tenants
NFR 5	Low cost, commodity parts	ESP32, DHT22, BH1750, relay board

4.2 Component architecture within the complete solution

The component follows a sense, decide and act loop distributed over four tiers (Figure 4.1). In the shade house, a **sensor node** in each section measures the air, and one **master controller** per house drives the pumps. In the cloud, a **Firebase Realtime**

Database [30] holds every reading, plan, command and acknowledgement, and a **FastAPI** [31] service on a rented virtual private server (VPS) runs the automation engine, the trained models and the analyses. The grower uses the shared **mobile application**, built with React Native [32], and is reached by alarms sent through **Firebase Cloud Messaging** (FCM) [33]. Within the complete solution, the component shares the application, the backend server, the database project and the account system with the other three components.

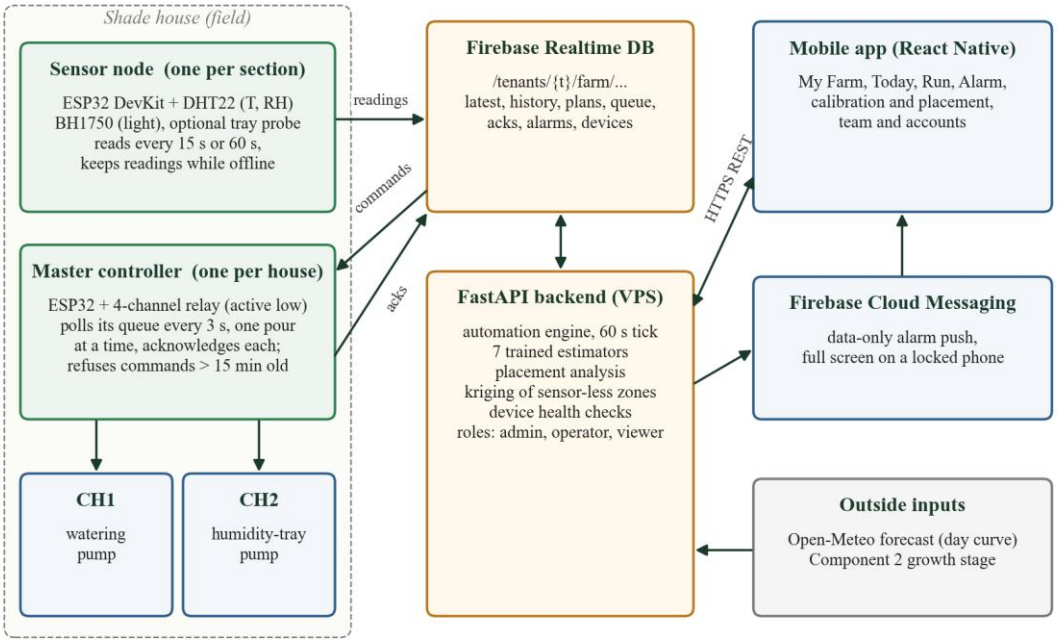


Figure 4.1: Architecture of Component 3 within the shared solution

4.3 Design alternatives and decisions

Table 4.2 summarises the main design decisions, the alternatives considered and the reason for each choice.

Table 4.2: Main design decisions

Decision	Alternatives	Chosen, and why
What the model predicts	Whether to water now, or when and how long	When and how long, because the grower waters daily anyway
What is measured	Probe on each plant's roots, or air in each section	Air, because every section can provide it and no plant is touched
Humidity tray amount	Measure the tray's level with a probe, or let the grower set the amount	Grower's amount (seconds, default one full tray), because a probe adds a fault mode and the volume is known
Who decides	Firmware decides on the node, or cloud decides and node confirms	Cloud decides, because models and rules can be tested and changed without reflashing, and firmware keeps a few hard limits and a copy of the day's plan for when the cloud is gone
Plumbing	One pump with a valve per section, or one pump per job	Two pumps per house, so a tray fill can never wet the roots
Model evaluation	Random split, or out-of-time and unseen years	Out-of-time, because adjacent days are correlated
Anticipation threshold	Maximise F1 (0.50), or favour precision (0.70)	0.70, because a false fill costs a six hour rest and a missed one is caught within the hour
Number of sensors	Fixed by design, or derived from calibration	Derived, never below four (kriging needs four)
Alarm delivery	Expo push service, or FCM direct with data only messages	FCM direct, because it needs one account fewer and data only messages reach a closed application

4.4 Detailed design

4.4.1 Sensor node and master controller

Every node is built on a 30 pin ESP32 DevKit [34], for its integrated Wi-Fi, low cost and mature toolchain. Temperature and humidity come from a DHT22 [35], whose

stated accuracy of ±0.5 °C and ±2 % RH sets the tolerance used in the placement analysis, and illuminance from a BH1750 [36] on the Inter-Integrated Circuit (I²C) bus (Table 4.3, Figure 4.2). A node carries no other sensor. Until 1 October 2026 a capacitive probe on general-purpose input/output (GPIO) pin 34 reported the humidity tray's water level. It was removed (Section 4.4.6), and GPIO34 is now unused.

Table 4.3: Pin assignments of the sensor node and master controller

Function	Part	ESP32 pin	Notes
Temperature and humidity	DHT22	GPIO4 (D4)	single wire data, 3.3 V
Illuminance	BH1750 (GY-302)	GPIO21 SDA, GPIO22 SCL	I²C address 0x23
Watering pump (master)	relay channel 1	GPIO25 (D25)	relay active low
Humidity tray pump (master)	relay channel 2	GPIO26 (D26)	relay active low
Fertilizer pump (reserved)	relay channel 3	GPIO27	specified, not built
Status light	onboard LED	GPIO2	blinks on Identify

Source: firmware/sensor_node_validate/sensor_node_validate.ino, validation-2.8

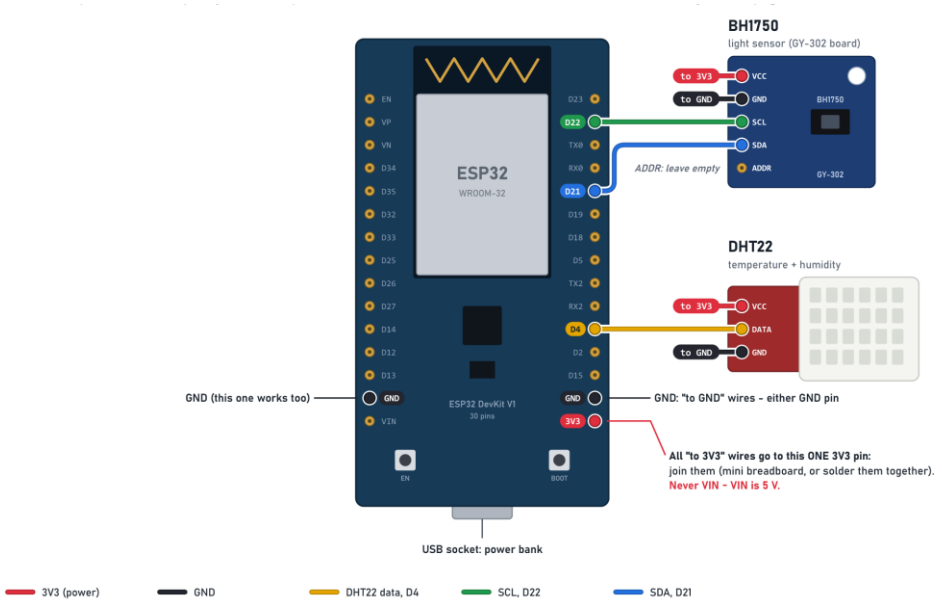


Figure 4.2: Wiring of a sensor node, generated from the firmware's pin map

The master controller is the same board with a four channel opto-isolated relay module (Figure A.1 in Appendix A). Channel 1 switches the house's watering pump and channel 2 the humidity tray pump, both from a 12 V supply. A pump that serves the whole house belongs to the house, and with two pumps a tray fill can never send water onto the roots. The relay inputs are active low, so the firmware drives every relay pin to its off level before configuring it as an output. The pump's delivery rate was measured on the bench on 28 August 2026 by timing pours into a jug on a scale. Five seconds gave 1.13 L and eight seconds 2.03 L, a rate of 0.300 L/s from the difference, which cancels the volume left priming the tube. Pour lengths are converted to litres with this rate, or with a per section rate where one has been measured.

A carrier printed circuit board was designed for the node and a first board made by a local workshop. Probed pin by pin on 2026, its ESP32 sockets were found not to connect to the sensor tracks, so the farm nodes are wired by hand (Figure A.2).

4.4.2 Node firmware

One firmware runs on every board in two builds. The **normal** build reads every 15 seconds, sends a heartbeat every 30 seconds and can act as a master. The **quiet** build, for nodes that only record, reads every 60 seconds and never polls for commands. The interval can be changed from the application within 5 seconds and 1 hour. Each cycle also performs blocking Hypertext Transfer Protocol Secure (HTTPS) requests, so the observed period is the setting plus about 11 seconds (27 s measured at 15 s, 71 s at 60 s). A reading carries temperature, humidity, illuminance, VPD, a fault flag and a timestamp from the node's clock, which it sets from the date in the reply of a Google web server. A sensor that does not answer is reported as -999, never as a default. The farm nodes run version 2.6. Version 2.8, which no longer reads or sends the tray probe and adds the offline fallback, is compiled and will replace it when the nodes are next on USB, and until then the backend drops the probe fields.

- **Self-registration.** A node registers its MAC address and fetches its assignment to a house and section, and an unassigned node uploads nothing.

- **Store-and-forward.** While the Wi-Fi is down, readings go to a buffer of up to 8,640 records in flash (LittleFS) and are uploaded oldest first, under their original timestamps, when it returns.
- **Liveness.** A heartbeat updates the node's last seen time independently of readings, and a backend ping is answered with the same token.
- **Wi-Fi setup from a phone.** After three failed joins, or with BOOT held for three seconds, the node raises its own hotspot, *OrchidNode-XXXX*, with a setup page at 192.168.4.1. The hotspot is held for ten minutes while the saved network is retried in the background.
- **Master command queue.** The master polls its queue every three seconds and runs one pour at a time. It refuses a command more than 15 minutes old or received before its clock is set, marks a pour as running before switching the relay, checks for a stop every five seconds, caps every pour at 120 seconds, and acknowledges each pour as done, stopped or refused with the seconds the relay was actually on.
- **Offline fallback.** The engine sends each master a copy of the day's plan whenever it changes, and a heartbeat every minute. If the database has not answered for ten minutes, or the heartbeat has stopped, the master waters each automatic section itself. It starts 25 minutes after the planned minute, when the engine's own 20 minute late window has closed, and never later than two hours after it. It waters a section at most once a day, counting waterings from the queue, needs a known clock, and uses yesterday's plan only when today's never arrived. It reports each such watering when the cloud is back, and the engine records it like any other.

4.4.3 Cloud service and data model

The backend is a FastAPI application behind the Caddy web server, which terminates TLS, on a VPS (Ubuntu 24.04, 4 cores, 8 GB) costing USD 6.60 per month. Farm data lives under the farm's tenant in the database. Each house has a metadata record (dimensions, master, pump channels, life cycle state), and each section its latest reading, today's plan, tray state, fertilizer state and control settings. The reading history sits under its own branch keyed by time ordered identifiers, and each master has its queue and acknowledgements (Table A.2 in Appendix A). The component exposes 69 representational state transfer (REST) endpoints. Every endpoint requires a signed-in

user and serves only that user's farm, every endpoint that changes anything requires a sufficient role, and a database write that fails is reported to the caller as a failure, never as a success.

4.4.4 Features and models

The vapour pressure deficit in kilopascals is computed for every reading from the temperature T in °C and the relative humidity RH in percent, using the FAO-56 expression [6].

$$VPD = 0.6108 \exp(17.27\ T / (T + 237.3)) (1 - RH / 100) \qquad (4.1)$$

The watering models take ten features, namely the four dawn readings (temperature, humidity, illuminance and VPD at 05:00), three summaries of the previous day (peak temperature, mean humidity, mean VPD), the month, the growth stage and the section's light exposure. The tray models work hour by hour. The dose model uses the five instantaneous values (temperature, humidity, illuminance, VPD, hour), and the anticipation model uses these, the month and eleven trajectory terms (the one, two and three hour changes in humidity, temperature and VPD, the one hour change in illuminance, and the three hour minimum humidity), all computed from readings the node has already sent. Seven estimators are deployed (Table 4.4).

Table 4.4: Deployed estimators

Module	Estimator	Inputs	Decides	Configuration
Watering time	Random forest regressor	10	start time, 06:00-09:00	50 trees, depth 16
Watering duration	Random forest regressor	10	pump seconds, 30-120	50 trees, depth 16
Tray fill	Random forest regressor	5	whether a fill is needed now	180 trees, depth 16
Tray anticipation	Random forest classifier	17	RH below 60 % within 3 h	60 trees, leaf ≥ 25, threshold 0.70
Feed due	Decision tree classifier	5	feed today, yes or no	depth 6, encodes a rule
Feed formulation	Decision tree classifier	5	nitrogen-phosphorus-potassium (NPK) ratio	depth 6, encodes a rule
Day ahead forecast	Random forest regressor	12	hourly T and RH, 06:00-18:00	50 trees, depth 16

Source: model bundles in ml_pipeline/results

The two watering forests were trained with 200 trees and trimmed to 50 to fit on the server, with their metrics recomputed on the same held-out data. The anticipation classifier's size was chosen by measurement (200 trees with a minimum leaf of 5 reached AUC 0.987, 60 trees with a leaf of 25 reached 0.986 at about a twelfth of the size), and its threshold of 0.70 favours precision because a false anticipation starts the tray's six hour rest while a missed one is filled within the hour by the reactive path. The tray regressor was trained to output pump seconds, but since the probe's removal only its fill or no fill call is used, and the amount is the grower's. The fertilizer trees reproduce a deterministic schedule and are kept in classifier form only so that they can be retrained on recorded grower decisions. The forecast combines the dawn reading with an outdoor forecast to draw the day's expected curve for the grower and changes no decision.

4.4.5 The daily automation engine

The engine is an asynchronous loop that wakes every 60 seconds and does whatever is due (Figure 4.3). It acts only on active houses, and a house that is calibrating is recorded but never watered automatically. All times are farm time (UTC+5:30). Every pass also writes a heartbeat, and whenever the day's plan changes the engine sends each master a copy of it for the offline fallback (Section 4.4.2).

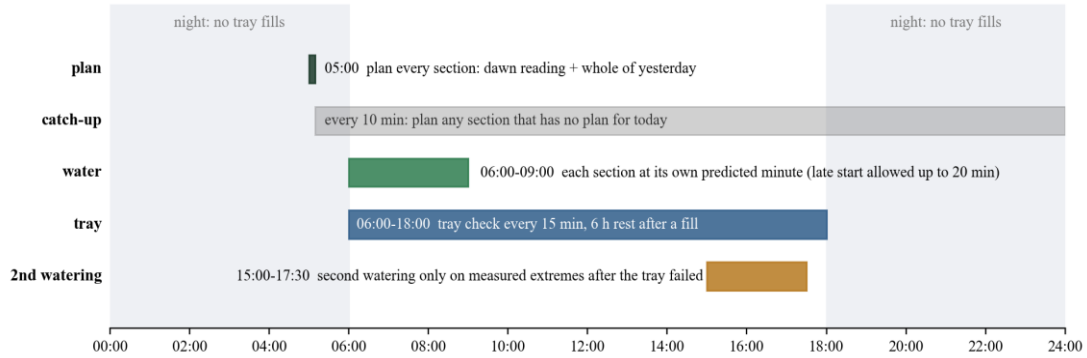


Figure 4.3: What the automation engine does over a day

- **05:00, the plan.** For every section the engine assembles the dawn reading and the whole of yesterday from the stored history and asks the watering models for a time and duration, and a plan built from fewer than 20 hours of yesterday says so. A section without its own sensor is planned from its kriged estimate and its neighbours' measured yesterday.
- **Catch up every 10 minutes.** Any section without a plan for today, such as one in a house activated after 05:00, is planned at the next catch up.
- **06:00 to 09:00, watering.** Each section is watered at its own predicted minute, or up to 20 minutes late if the server was down then, never later. The day counts as watered only when the master acknowledges the pour.
- **Every 15 minutes in daylight, the trays** (Section 4.4.6).
- **15:00 to 17:30, a second watering, only on measured extremes.** A lighter watering (70 % of the morning's) is allowed only when the morning watering ran, the latest reading is at most 30 minutes old and shows at least 36 °C or a VPD of at least 3.0 kPa with humidity below the band, and the tray has been filled and has still failed to hold humidity up.

Some rules hold whatever a model predicts. No plan, watering or tray decision is made from a reading more than two hours old or a kriged estimate more than an hour old, readings outside physical limits are rejected, and a failed reading is dropped, never replaced by a default. Whether the engine acts or only advises is set by one **automatic care** switch per farm, which each section can follow or override. When it is off, the engine makes the same decisions and sends the grower an alarm instead. If a house has no master or no pump channel, nothing is recorded as watered and an alarm names what is missing.

4.4.6 Humidity tray loop

Each section has a shallow tray of water under the plants (32 cm by 48 cm, 3 cm deep with an overflow notch, 4.6 L full) whose evaporation raises the humidity around the plants without wetting the roots, and it is filled by the house's tray pump. Every 15 minutes between 06:00 and 18:00, for each section with its own sensor, the models decide **when** the tray is filled, from the tray regressor's call that a fill is needed now (when humidity is below the band, or inside it on a hot, bright day) or the anticipation classifier's prediction that humidity will fall below 60 % within three hours. The grower decides **how much**. Every fill pours the section's tray amount, set in seconds on the section's Pour lengths card (1 to 15 seconds), or one full tray (15 seconds, about 4.5 L at the measured rate) when none is set. After a fill the tray rests for six hours, shortened in proportion when the grower's amount is less than a full tray. Humidity still below the band within that rest means the air, not the tray, is the limit, which is the condition the second watering rule waits for. A tray is never filled at night or into air already above the band.

Until October 2026 a capacitive probe in the tray reported its water level, and the loop refilled an empty tray and held off a full one. It was removed. Its readings needed a fault handling path of their own. An unplugged or floating probe reads as a perfectly plausible empty or full tray. On the farm one read 0 % for thirty hours while the system, seeing humid air, reported that no fill was needed. The loop also had to test whether the tray "answered" a fill before trusting it. The grower, meanwhile, knows the volume of the tray and the pump. Making the amount a setting and leaving only the timing to the models removed a sensor, a wire and a class of false alarms from every node.

4.4.7 Fertilization scheduling

A feed of 30-10-10 (high nitrogen) is scheduled during active growth and 10-30-20 (high phosphorus) during flowering, every 7 days at half strength, and every 14 days at quarter strength from November to February. A dormant plant is never fed. This is a hard rule in the backend, added after an earlier model revision was seen proposing feeds during dormancy. The growth stage is the grower's setting for the section, chosen when the farm is set up (a house created through calibration starts as Active) and flagged for a check when it is more than 60 days old. Where none has been set, the backend guesses from the month and asks the grower to confirm. A due feed is attached to the section's next watering, so it always reaches wet roots, and is recorded as given only when the master acknowledges the pour. If the master refuses it, the feed returns to due and the grower is told. The dosing pump is not built. Today the grower mixes the feed into the reservoir when the application says it is due, and relay channel 3 is reserved for a doser.

4.4.8 Calibration and sensor placement

A new house starts from an even spread of sensor positions over its dimensions (Figure 4.5a) and enters calibration with one node per section. The analysis is a flow of eight steps (Figure 4.4). Each step produces its own numbers, which the backend stores with the analysis, and the application shows the chosen placement, one node's reach drawn on the house plan, and the error scores.

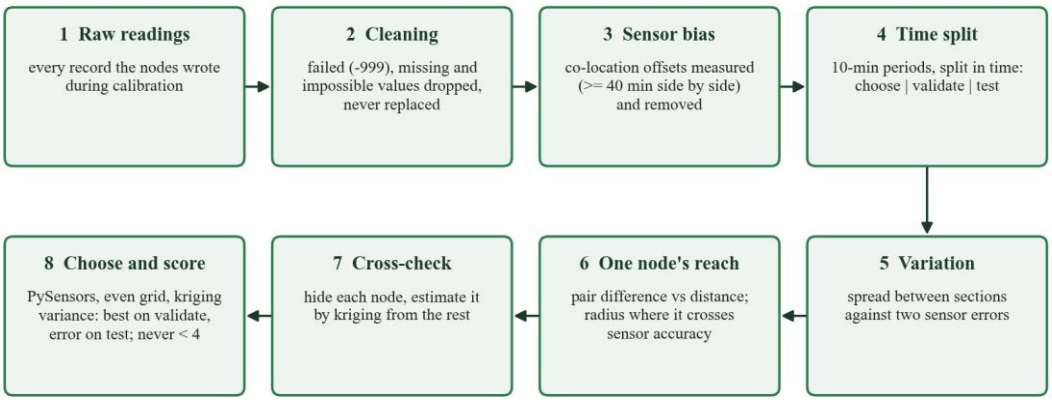


Figure 4.4: The eight steps of the placement analysis

(1) **Raw readings.** Every record written by the house's nodes during calibration, with nothing generated.

(2) **Cleaning.** Failed (-999), missing and impossible values are dropped and counted, never replaced.

(3) **Sensor bias.** If the nodes first ran side by side for at least 40 minutes (co-location), each node's offset from the group is measured and removed. Otherwise the step reports that the bias was not measured.

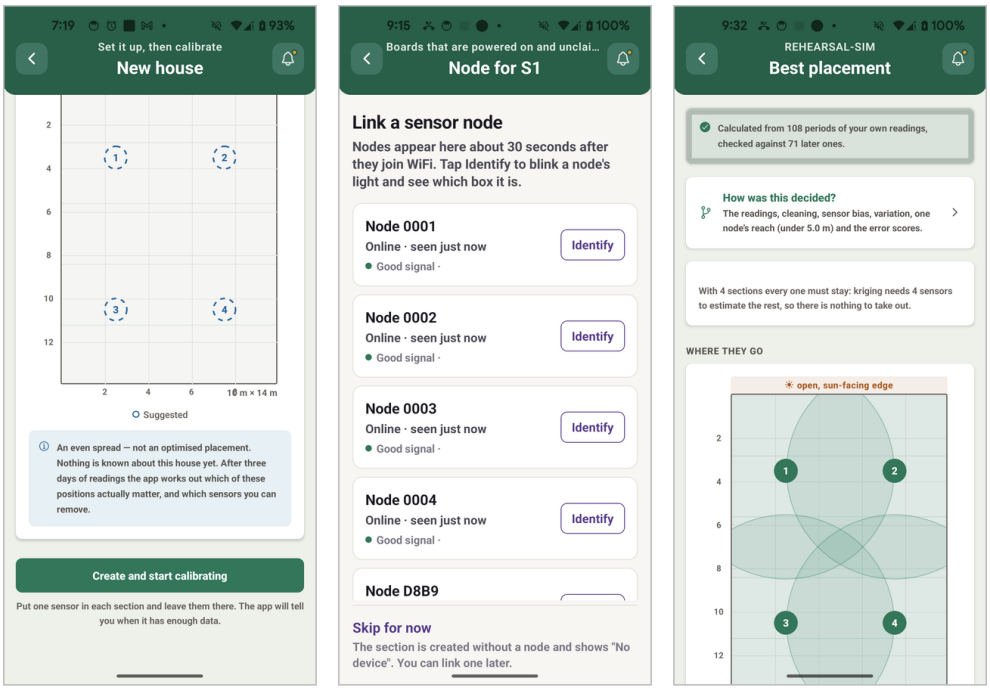
(4) **Time split.** Readings are averaged into 10 minute periods in which every node reported (at least 24 in common) and split in time order into *choose*, *validate* and *test* parts, the last 40 % held out.

(5) **Variation.** How far apart the sections are for temperature, humidity and VPD, against the combined error of two sensors.

(6) **One node's reach.** For every pair of nodes, the mean difference is set against distance. The distance at which the fitted line reaches the sensor's accuracy (0.5 °C, 2 % RH, 0.10 kPa) is the coverage radius, or an upper bound when the house varies with direction.

(7) **Cross-check.** Each node is hidden in turn and estimated from the others by kriging.

(8) **Choose and score.** PySensors (SSPOR) [20], an even grid and a greedy kriging variance method each choose which sections keep a sensor. The best on the *validate* part is scored on the *test* part against six random layouts and, where there are at most 35 possible layouts, every one of them. The system never keeps fewer than four sensors.



(a) even spread (b) link a node (c) best placement

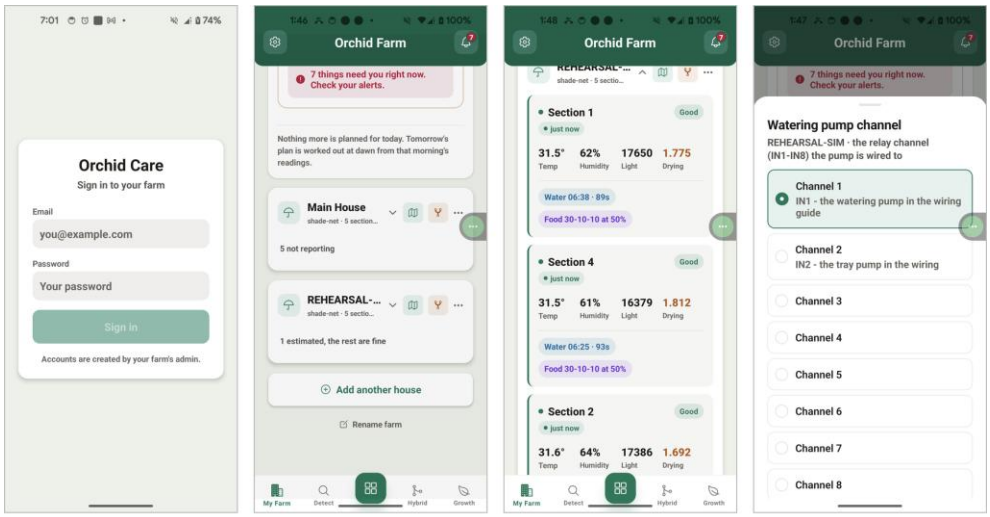
Figure 4.5: Calibration in the application: (a) the even spread proposed for a new house, (b) linking a node to a section, (c) the chosen placement with one node's reach drawn on the plan (screens (b) and (c))

4.4.9 Estimating sections without a sensor

Every five minutes the engine estimates each section without a node from the sections that have one, by ordinary kriging over their positions, falling back to inverse distance weighting with a power of 2 when no variogram can be fitted. With fewer than four measured sections no estimate is made. An estimated section is labelled as estimated everywhere in the application, planned from its estimate and its neighbours' measured yesterday, and watered through the master like any other, but tray decisions need a section's own sensor.

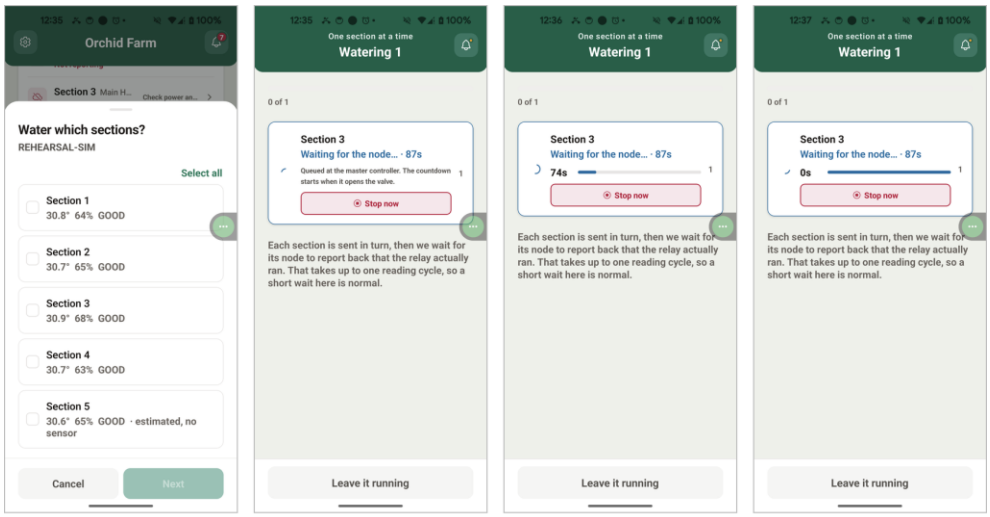
4.4.10 Mobile application

The application offers a simple *Today* screen with large text, plain words and one button, and an expert *My Farm* dashboard with readings, VPD, plans and controls (Figure 4.6). Accounts are created by the farm's administrator, and public sign-up is off. An *admin* manages users and settings, an *operator* may water and fill trays, and a *viewer* can see the farm but never start a pump. Watering by hand goes through the master's queue, and the screen counts down only once the master has opened the valve and reports the seconds actually run (Figure 4.7).



(a) sign in (b) My Farm (c) sections and plans (d) pump channels

Figure 4.6: The application: (a) sign-in, (b) the farm with its houses, (c) sections with readings and today's plan, (d) choosing a house's pump channels



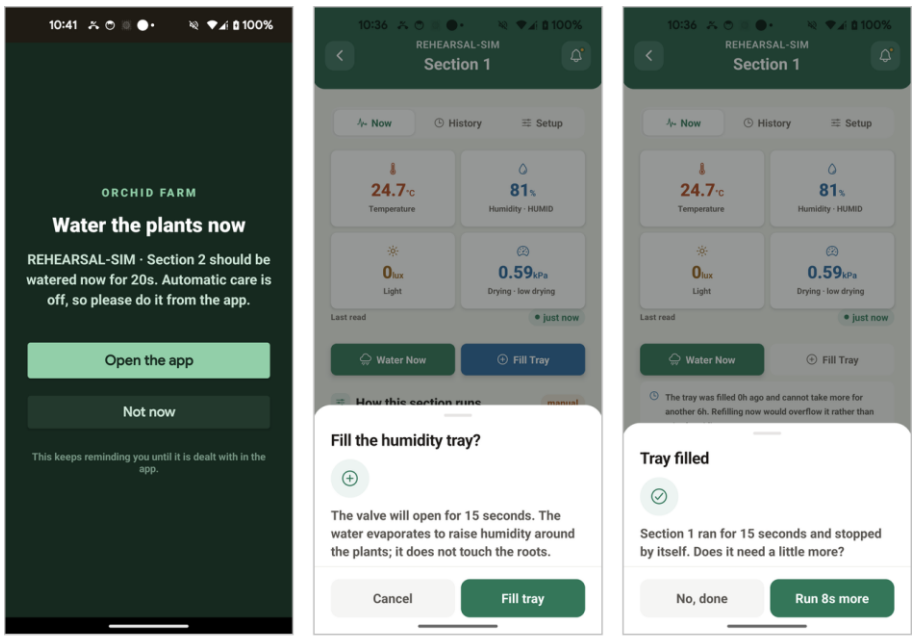
(a) pick sections (b) queued (c) master running (d) finished

Figure 4.7: Water Now: (a) choosing sections, (b) queued at the master, (c) the master running the pour, (d) finished and confirmed by the master

4.4.11 Alarms and device health

When something needs a person, the backend sends a data-only alarm through Firebase Cloud Messaging, and the application's native service shows it full screen over the lock screen, at alarm volume, with a button that opens the application on that alarm and its one relevant action (Figure 4.8). An alarm repeats every five minutes, up to six times, until someone acts on it or acknowledges it. The system also checks its own hardware. A temperature sensor failing three readings in a row, a light sensor silent for ten minutes, a node silent for twenty, a frozen or jumping reading, and a pour the

master never carries out each raise an alarm. Freshness is judged against each node's own reading interval, within limits of 1.5 and 20 minutes.



(a) full-screen alarm (b) fill the tray (c) tray filled

Figure 4.8: An alarm on a locked phone (a), the action it leads to (b) and its confirmation (c) (test house)

4.5 Standards and professional practice

- **Engineering safety.** Relay boards are opto-isolated and active low with pins driven off before use, every pump carries a flyback diode, and sensors run at 3.3 V with nothing but the relay board on the 5 V pin. Physical limits (120 second pours, tray capacity, no stale commands) are enforced in both firmware and backend.
- **Security.** TLS on every connection, signed-in access only, role based authorisation checked by the server, tenants isolated in the database, secrets kept out of the public repository, and the remaining open gap is stated (Section 3.6).
- **Software practice.** Version control with Git and pull requests, continuous integration on every push, one regression test written for every defect found, and every write checked and its failure reported.
- **Evidence.** Every reported number is produced by a script and read from its output. Generated, rule based and measured data are labelled as such, and references follow the IEEE style recommended by SLIIT.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Table 5.1 lists the technologies used in each layer of the component.

Table 5.1: Technologies used by the component

Layer	Technology
Sensor node and master	ESP32 DevKit (30 pin), DHT22, BH1750, 4 channel relay, Arduino core for ESP32, ArduinoJson 7, LittleFS and arduino-cli
Database	Firebase Realtime Database (REST from the nodes and the backend)
Backend	Python, FastAPI, Uvicorn, Caddy, Ubuntu 24.04 VPS, and firebase-admin for authentication and messaging
Machine learning	scikit-learn 1.5 (random forests, decision trees), pandas and NumPy, with ERA5 through Open-Meteo
Spatial methods	PyKrige 1.7.2 (ordinary kriging), python-sensors 0.4.1 (PySensors SSPOR)
Mobile application	React Native with Expo SDK 54, a native Android (Kotlin) alarm service and activity, and Gradle release builds
Notifications and accounts	Firebase Cloud Messaging (data only messages) and Firebase Authentication with custom claims
Testing	pytest, Node test runner, GitHub Actions, and a browser based digital twin and node simulator

5.2 Component implementation

Table 5.2 gives the size of the component's code. The backend's core is the care module (setting up houses and sections, planning, watering, the tray loop, fertilization, calibration) with the automation engine, the device registry and health checks, the placement analysis and the kriging service. The firmware is one sketch with tabs for the master queue, store-and-forward and the offline fallback, and the application adds eighteen screens to the shared application.

Table 5.2: Size of the component's code

Part	Files	Lines
Backend routes and services of Component 3	16	10,582
Mobile screens of Component 3	18	9,445
Node firmware	3	2,544
Backend tests	30	6,459

Source: line counts of the repository, 10 Oct 2026

5.3 Integration with the other components

Component 3 shares the mobile application, the backend server, the Firebase project and the account system with the other three components. It does not depend on any of them for its decisions. The growth stage that selects the feed is set by the grower, a choice made at the planning stage so that the watering component can run on its own. The account system built for this component provides sign-in for the whole application, and the backend server set up for it hosts all four components' services. Components 1 and 4 exchange no data with Component 3.

5.4 Verification and software testing

5.4.1 Automated tests

The backend's test files cover the behaviour most likely to cost water or trust if it regressed, namely freshness against each node's interval, the farm clock, the automatic care switch, alarm repetition, authentication and roles, isolation of one farm's data from another's, the placement analysis, device health and liveness, master commands, the humidity tray amount, the offline fallback, and one test for each defect found in the end-to-end campaign. Two tests check that every control the application shows is gated by the permission the server enforces. Results are given in Section 6.2.

5.4.2 Simulation before hardware

A **digital twin** in the browser writes readings to the real database on the nodes' own paths, carries out commands and models tray evaporation, at up to ten simulated minutes per second. It exposed defects unit tests had not. Automatic mode diagnosed empty trays and never filled them, the tray's rest was timed on the server's clock instead

of the device's, screens never refreshed while open, and the first planner split two shaded spots differing by 0.2 % per hour. A **node bench** of virtual ESP32 boards rehearses registration, linking and Wi-Fi changes, and a **rehearsal harness** drove four virtual nodes with deliberately injected sensor biases through the real application to test calibration.

5.4.3 Hardware bench tests

- **Command to pump.** A 3 second tray command issued from the backend was acknowledged by the reference node as done 18 seconds later, one reading cycle at the 15 second setting.
- **Relay safety.** The relay was confirmed active low, and no pump starts during boot.
- **Pump delivery.** 5 s gave 1.13 L and 8 s 2.03 L into a jug on a scale, a rate of 0.300 L/s.
- **Liveness.** Successive heartbeats came 19 to 34 seconds apart, and pings were answered in 5.4 to 8.3 seconds, 18.5 seconds at worst.
- **Wi-Fi setup hotspot.** With firmware 2.6, a second ESP32 acting as a phone saw the hotspot appear and stay up, joined it, loaded the setup page (HTTP 200, twelve networks listed), saved the farm network, and saw the node join it.
- **The first carrier board.** A probing sketch found every sensor connection floating, so the board was set aside and the four farm nodes wired by hand, each confirmed reading its DHT22.
- **Firmware 2.8.** The version without the tray probe and with the offline fallback compiles in both builds. The fallback's decision was tested on a PC with 18 checks, each of three deliberate faults put into it was caught, and the backend's side has 13 automated tests. It has not yet been run on a board or flashed to the farm nodes.

5.4.4 End-to-end test campaign

On October 2026, nineteen scenarios covering calibration, choosing a master, Water Now and Stop, tray fills, scheduled watering with automatic care on and off, feeds, repeated alarms, every device health fault, a Wi-Fi outage, an offline master with a stale command, the full screen alarm path, and the planning and watering of a section without a sensor were run on the deployed backend and the real application against a farm house (Table A.1 in Appendix A). Results are given in Section 6.2.

5.5 Deployment and operation

The backend tracks the main branch and is deployed by pulling it on the server and restarting the service, never between 06:00 and 09:00 so that a restart cannot fall in the watering window. The Android application is built as a release package with clear text traffic permitted in the native manifest, checked on every build, and installed directly on the grower's phone. Firmware is compiled with Arduino cli and flashed over USB.

5.5.1 Field deployment

On October 2026 the system was installed in a shade net house on the farm, registered as house H1 "Test", 6m by 3m, with four sections and one node each, connected through a portable 4G router.

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 Watering time and duration

Table 6.1 and Figure 6.1 give the error of the watering models under the three designs. Trained on 2017 to 2022 and tested on 2023 and 2024, the watering time model is off by 8.42 minutes on average (R^2 = 0.954). Trained on all eight years and tested on weather from 2025 and 2026 downloaded after training, it is off by 8.41 minutes (R^2 = 0.952). The duration model's error is 2.54 seconds out of time and 2.51 seconds on unseen weather.

Table 6.1: Error of the watering models under three evaluation designs

Design	Train	Test	Test rows	Time MAE (min)	Time R^2	Duration MAE (s)	Duration R^2
Random 80/20	2017-2024, 80 %	2017-2024, 20 %	11,688	8.45	0.951	2.53	0.962
Out-of-time	2017-2022	2023-2024	14,620	8.42	0.954	2.54	0.964
Unseen	2017-2024	2025-2026	2,364	8.41	0.952	2.51	0.964

Source: backend/app/data/watering_validation.json, written by ml_pipeline/validate_out_of_time.py

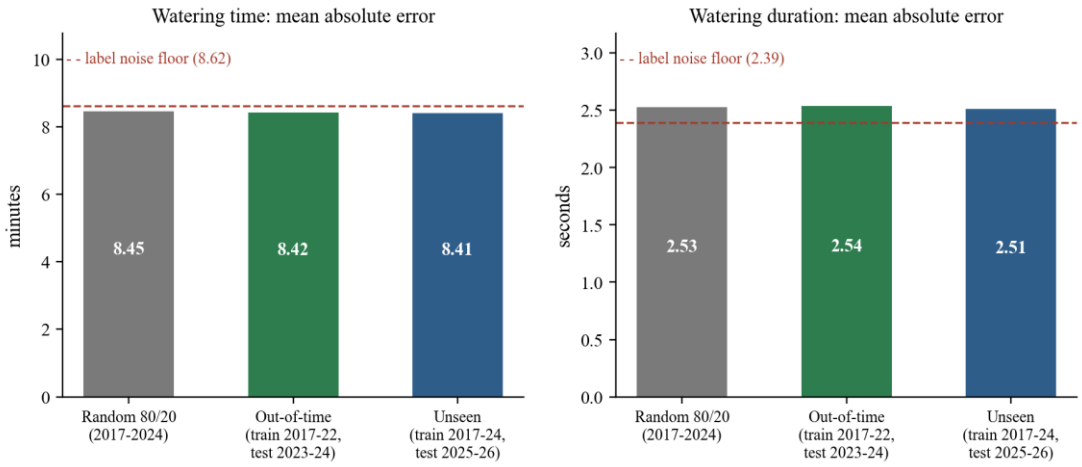


Figure 6.1: Mean absolute error of the watering models under the three designs, against the noise built into the labels

Withholding whole years changes the error by hundredths of a minute, so the model learned a relationship that holds on unseen weather, not the days it was trained on. The error also sits at the labels' noise floor (8.62 minutes and 2.39 seconds), and the time error is slightly below its floor because the floor is computed before the labels are clipped to the window. Against these labels the error cannot be reduced further, because the model reproduces the expert rule on unseen years.

6.1.2 What the watering models use

Yesterday's mean VPD carries 0.949 of the importance for the watering time and 0.888 for the duration, while the four dawn readings together carry 0.010 and 0.008, and dawn light nothing measurable (Figure 6.2). The section's exposure carries 0.071 of the duration model's importance and almost none of the time model's.

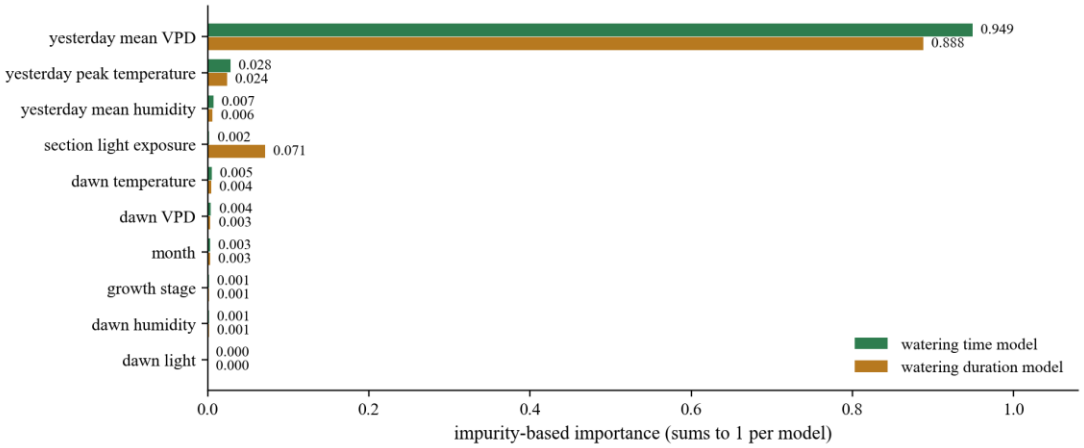


Figure 6.2: Feature importance in the deployed watering-time and duration models

Source: feature_importances_ of the forests in watering_v2.pkl

These importances reflect how the labels were made. The rule uses only yesterday's summaries and, for the duration, the exposure, so a model that learned it should ignore the dawn readings, and it does. Exposure matters to the duration and not to the time, exactly as in the rule. The analysis is therefore a check that the models learned the rule's structure, not a discovery about orchids. The dominance of VPD over the other two summaries, which the rule weights equally, is better explained statistically. VPD is computed from temperature and humidity and correlates with both, and impurity importance concentrates on such variables [12]. The design conclusion it supports is that a day's plan depends on yesterday's record, which is why the engine plans from the whole of yesterday's stored history.

6.1.3 Humidity tray

The anticipation model was re-scored from the deployed file on the 47,515 daylight hours of 2023 and 2024, none seen in training, and reproduced the stored figures, with an area under the ROC curve of 0.986 (Figure 6.3, Table 6.2). A fall below the band followed within three hours in 9.1 % of daylight hours. At the deployed threshold of 0.70, 91.5 % of its early fills are followed by a real fall and it catches 64.8 % of falls in advance, and the rest are filled by the reactive path once humidity is measurably low. The threshold maximising F1, 0.50, would catch 77.3 % at the cost of 15.2 % false fills instead of 8.5 %. Of the six most important features, 4 describe the trajectory of the section's climate, namely humidity (0.237), VPD (0.160), 3 hour minimum humidity (0.111), 3 hour change in VPD (0.098), 2 hour change in VPD (0.090), 1 hour change in VPD (0.066). The model fires while humidity is still in band but falling steeply, the case a threshold controller cannot see.

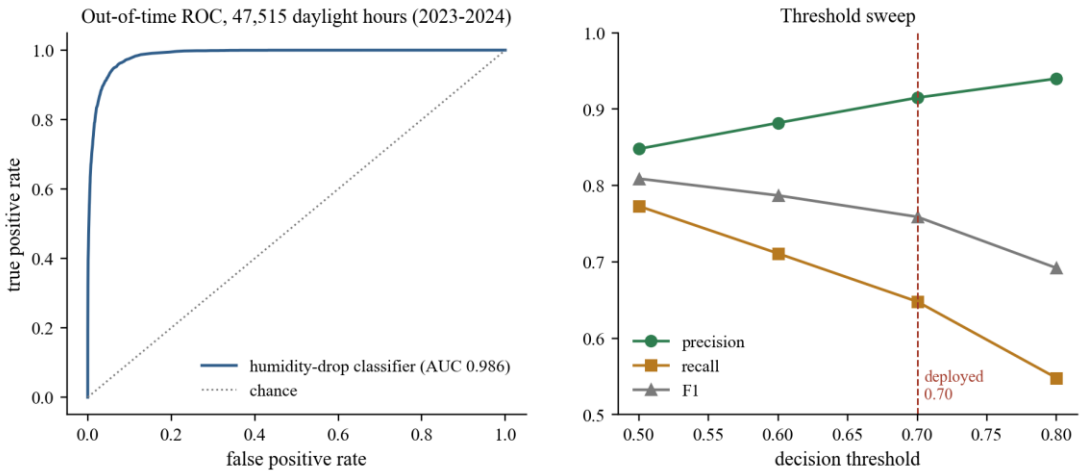


Figure 6.3: Out-of-time performance of the humidity-drop classifier: ROC curve and threshold sweep

Table 6.2: Precision, recall and F1 of the humidity-drop classifier at four thresholds (2023 to 2024)

Threshold	Precision	Recall	F1	Note
0.50	0.848	0.773	0.809	highest F1
0.60	0.882	0.711	0.787	
0.70	0.915	0.648	0.759	deployed
0.80	0.940	0.548	0.692	

Source: deployed tray_v3.pkl re-scored read-only on 10 Oct 2026

The tray regressor matches its training rule with a mean absolute error of 0.337 seconds. Since the probe's removal only its fill or no fill call is used, this figure describes the timing signal's source rather than the amount poured, which is the grower's.

6.1.4 Fertilization and forecast

The feed due and formulation trees score accuracy and F1 of 1.000 on held-out data. This is not reported as a result, because the labels are a deterministic function of the days since the last feed and the growth stage. The forecast predicts the hourly temperature from 06:00 to 18:00 with a mean absolute error of 0.50 °C ($R^2 = 0.915$), humidity with 2.22 %, the day's peak with 0.443 °C, and a hot day (33 °C or more) with F1 = 0.917. Because reanalysis actuals stood in for the outdoor forecast during training, these are an upper bound. They were computed on the full forest before trimming and could not be recomputed because its held-out split was not saved.

6.1.5 Placement method on data with a known answer

- **Sensor bias.** A bias of +0.4 °C and −2.0 % RH injected into one node during co-location was recovered as +0.383 °C and −2.015 %.
- **Reach of one node.** Six nodes on a line in a house warming by 0.1 °C per metre should give a radius of 5 m, and the analysis measured 5.03 m.
- **Direction dependent houses.** Four nodes in a 2 × 2 layout over the same gradient gave the upper bound "under 5.0 m" rather than a single radius.
- **The full flow in the real application.** In the rehearsal with four virtual nodes, injected temperature biases were recovered within 0.025 °C and humidity biases within 0.17 % RH.

6.2 Product and technical results

Automated tests. Run on October 2026, after the tray probe was removed, the backend's 30 test files collected 368 tests. Of these, 336 passed and 31 were skipped, the skipped ones being integration tests that need the live service and its key. One failed, and only when every file runs in one process, because an earlier test leaves Python's event loop closed and this one asks for the current loop. It passes on its own,

and the 304 tests of the continuous integration job, run locally in that job's order, all passed. The failure is a test harness defect and is recorded as an open item.

End-to-end campaign. All nineteen scenarios passed once the defects they exposed had been fixed. Table 6.3 gives the measured response of the deployed system, and Table 6.4 the defects found.

Table 6.3: Measured response of the deployed system in the end-to-end campaign

Event	Measured
Water Now command to master	received 3 s after issue and acknowledged as done after 45 s
Stop during a pour	acknowledged as stopped, 15 s run
Scheduled watering with automatic care off	alarm at the planned minute, phone notified within 1 s
Unacknowledged alarm	repeated after 5 min 8 s
Temperature sensor fails	alarm 3 min 17 s after the first failed reading
Light sensor fails	alarm 11 min 27 s after the first failed reading
Node goes silent	alarm 21 min after its last reading
Wi-Fi outage at a node	18 buffered readings uploaded on return, in time order
Master offline, Water Now queued	alarm after 19 min 20 s, and on return the 1,395 s old command was refused as stale with nothing poured
Fed watering refused as stale	feed returned to due 82 s later and grower told the plants were not fed
Section without a sensor	estimated, planned from its four neighbours, offered in Water Now

Source: Rehearsal/E2E_RESULTS.md, 8 and 9 Oct 2026

Table 6.4: Defects found by the end-to-end campaign, by severity

Severity	Found	Fixed	Examples
Critical	1	1	F15 sections without a sensor never planned or watered
High	9	9	F4 Water Now with no master reported success, F17 failed writes reported as saved, F18 plan read the night as yesterday, F20 refused pour kept its feed
Medium	6	6	F8 house activated after 05:00 got no plan that day, F16 tray fill sent at 02:02
Low	9	9	F23 countdown ran before the valve opened, with F24 (grouped alarm omits the house name) and F25 (Settings shows the first online board) still open
Total	25	25	

Source: Rehearsal/E2E_RESULTS.md

Several of these would have caused real harm on a farm. The most severe (F15) left sections without their own sensor unplanned and unwatered, because the planner required a section's own reading, and on a house where calibration had removed sensors, those sections would have gone dry. Others gave false assurance. A Water Now on a house without a master reported success, logged a watering and recorded a feed while no water moved (F4), a refused pour still counted its feed (F20), and the plan's "yesterday" was the last 2.6 hours of night (F18).

6.3 User or stakeholder validation

The system was evaluated with the owner of the orchid farm after testing the hardware and application on site for two weeks. Following this trial, an interview was conducted to get direct feedback on daily use, time savings, and practicality. The grower explained that watering the shade houses by hand normally takes about three and a half to four hours every day. Managing watering for the sections from one system cut out this daily manual work and saved significant time. He noted that growers normally guess moisture and heat just by looking around, which is not consistent, so having exact numbers and automatic control makes daily care much easier. He also welcomed the automated fertilization feature, noting that delivering feeds with the watering saves

extra labour and ensures plants receive nutrients regularly. The grower also highlighted the value of keeping a continuous climate history. General weather forecasts do not show what happens inside a shade house, and growers do not write down daily numbers. Having stored records of past temperatures, humidity drops, and waterings allows a grower to look back. Such as during hotter months and see how the shade house actually behaved. Finally, the grower praised the hardware design, noting that the IoT device is low cost and does not use expensive or complicated gadgets. Because small commercial growers cannot invest in costly equipment, keeping the setup simple, cheap, and effective makes it practical for real farm adoption.

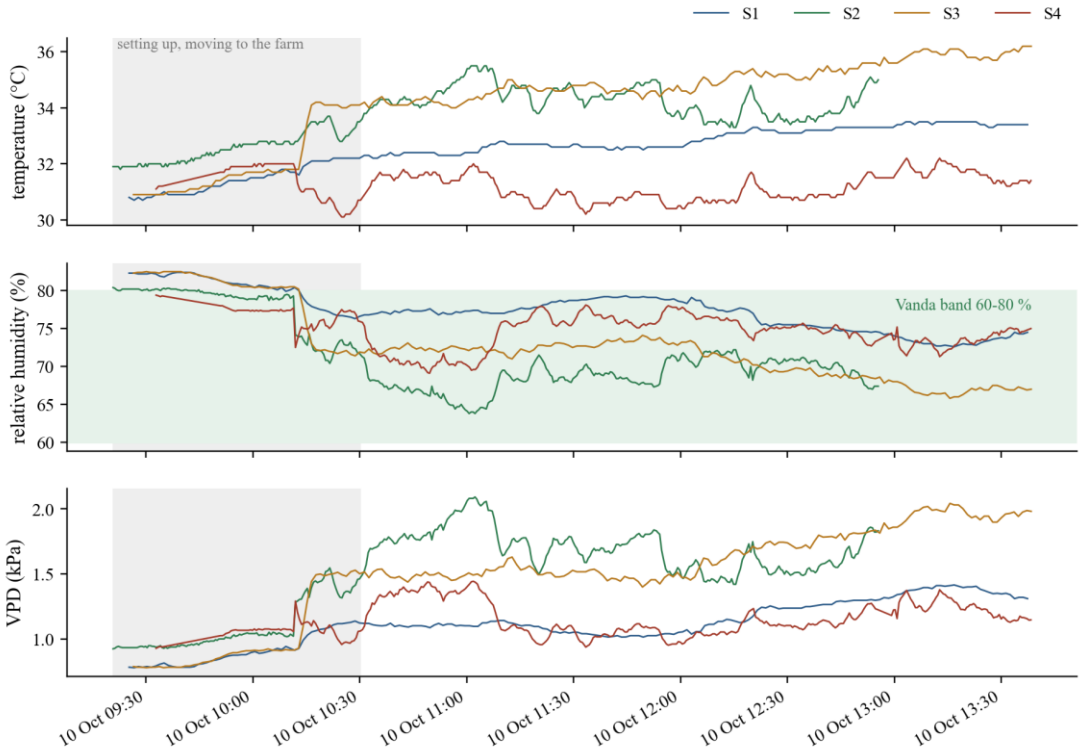


Figure 6.4: Temperature, relative humidity and VPD recorded by the four nodes of house H1 on 1 Oct 2026 (grey: set-up time, excluded)

Source: readings downloaded from the farm's database

Table 6.5: Readings per section in house H1, 1 Oct 2026 10:30 to 13:38

Section	Readings	Mean T (°C)	T range (°C)	Mean RH (%)	RH range (%)	Mean VPD (kPa)
S1	153	32.9	32.2 to 33.5	76.4	72.6 to 79.3	1.18
S2	261	34.3	33.3 to 35.5	68.8	63.8 to 72.2	1.69
S3	154	35.0	34.0 to 36.2	70.5	65.8 to 74.1	1.66
S4	336	31.2	30.2 to 32.2	74.3	69.1 to 78.1	1.17

Source: farm_summary.json, computed from the farm's stored readings

Over the 15 ten minute periods in which all four nodes reported, the warmest and coolest sections differed by 3.84 °C on average (at most 4.36 °C) and their humidity by 8.63 % (at most 12.01 %). Two DHT22s in the same air may disagree by up to 1 °C, and the spread exceeded that in 100 % of the periods. Without co-location each sensor's bias is unmeasured, but a spread several times the worst case for two sensors is not plausibly sensor error. Section S3 was the warmest and S4 the coolest, and humidity stayed inside the 60 to 80 % band throughout. These are hours, not days. The placement analysis, which opens after a day of readings in every section, will decide how many sensors the house keeps.

6.4 Integration results

The shared parts of the solution work together in daily use. The component's screens, sign-in and alarms run inside the one application the group ships, and the one backend server hosts every component's service. Component 3 takes no input from the other components, because the growth stage that selects the feed is set by the grower in the application.

6.5 Summary of findings

(1) **Watering time and duration can be predicted from air conditions alone, and the prediction holds on unseen years** (RQ1), with 8.42 minutes out of time,

8.41 minutes on 2025 to 2026 weather, at the labels' noise floor. What is shown is that the model reproduces an expert rule on real, unseen weather.

(2) **A day's watering is set by the previous day's evaporative demand.** This follows from the rule rather than being discovered, but it is the design conclusion that the plan depends on yesterday's full record.

(3) **Humidity can be kept in band by acting early, from the section's own recent readings** (RQ2), with AUC 0.986 on two unseen years, with 91.5 % of early fills justified at the operating point.

(4) **A sensor that only adds a fault mode should go.** The tray probe needed its own fault handling and once misled the system for thirty hours, and a grower set amount with model decided timing does the same job with less hardware.

(5) **The number of sensors can be measured** (RQ3), as the method recovers known biases within 0.02 °C and a 5 m radius within 0.03 m, and the first hours on the farm show sections several sensor errors apart.

(6) **Confirmation by the hardware is what makes automation trustworthy** (RQ4), since most serious defects were not wrong predictions but wrong reports, removed by making the controller's acknowledgement the only evidence that water moved.

7 DISCUSSION

7.1 Interpretation of findings

The first question was whether the watering time and duration can be predicted from air conditions alone. The results say they can, with one condition attached. The models reproduce the watering rule on years they never saw. No record exists of when a skilled grower would have watered each section on each of thousands of days, so I wrote down what the horticultural guidance implies and generated the targets from real weather. A model trained this way cannot be better than its rule. What I can claim is that the predictions are consistent, can be explained, and stay inside the 06:00 to 09:00 window. I cannot claim that the rule is the best watering for the plants. Every watering the system makes is now recorded with the conditions behind it and the seconds the pump ran, so growers' corrections and plant outcomes can replace the rule later without changing anything else in the system.

The tray results are the stronger ones, because the anticipation target is the converted humidity series itself and not a rule. An AUC of 0.986 on two unseen years shows that a section's own recent climate carries the warning. I set the threshold at 0.70 instead of 0.50 because a wrong early fill costs six hours of rest, while a missed one is caught within the hour. Removing the tray probe made the division of work clearer. The models decide when, since only they see the trajectory of the air, and the grower decides how much, since the grower knows the tray.

The placement method recovers the answers it was tested against and is running on house H1.What the first hours do show is that the four sections differ by several degrees, which is the condition under which placement matters at all.

The fourth question is where the end to end campaign carries the most weight. The system detects its own faults within minutes, refuses to act on old information and reports what actually happened. Most of the 25 defects it found were not wrong predictions but wrong reports, such as a watering recorded when no water moved. That is why the controller's acknowledgement is the only evidence of a pour that the system accepts.

7.2 Comparison with literature baselines and existing solutions

The orchid irrigation studies I reviewed report water savings against manual practice [21], [23]. I have not measured water saved, because the deployment on the farm is days old. What this work adds is scheduling for bare root plants, where their substrate based controllers cannot be used, together with an anticipatory humidity loop, feeding only with water and a measured sensor count (Table 2.1). Against the first design, whose threshold rule scored F1 = 0.017 on its own synthetic labels, the more telling baseline was a model trained on synthetic weather. It answered 06:00 on every day, while the model trained on real weather spreads its times across the window with the conditions (Figure 3.1). Fauzy et al. [22] predict humidity to drive misting, and the same idea is applied here hours ahead, from the section's own trajectory and without an external service.

7.3 Research contribution

The research contribution has three parts. The first is a reformulation of *Vanda* irrigation as per section scheduling of when and for how long, validated on years held out of training and on weather obtained after training. The second is an anticipatory humidity controller driven by the section's own trajectory, with an operating point chosen by the cost of each error. The third is a measured, testable way to decide how many sensors a shade house needs and how far one sensor reaches. I have also tried to state exactly what rule based labels can and cannot show, including that the dominant feature follows from the rule.

7.4 Product contribution

The product is a working system on a real farm, with four nodes, a cloud service and the grower's application, and both automatic and manual operation. Every pour is confirmed by the hardware, safety rules are enforced in both firmware and backend, the humidity tray needs no level sensor, and sensors, nodes and the master are diagnosed by the system itself, with alarms that reach a locked phone, and a master can water from its copy of the day's plan when the cloud is gone.

7.5 Limitations and threats to validity

Table 7.1 lists the limitations, their effect on the results and what would resolve each.

Table 7.1: Limitations of the work

Limitation	Effect	What would resolve it
Labels from an expert rule, not grower decisions	The models can at best reproduce the rule	A season of recorded waterings, corrections and plant outcomes, then retraining
Node power use unmeasured, first carrier board unusable, farm nodes still on firmware 2.6	Battery life unknown, nodes on hand wiring, probe fields still sent	Measure current, remake the board, flash 2.8
No formal usability study	Ease of use for growers is asserted, not measured	A structured study with growers

7.6 Implications

For growers the system replaces one routine for a whole house with a plan for each part of it, and gives them a record of what was done. For researchers it shows that microclimate level scheduling is possible without substrate sensing, and that the number of sensors can be measured instead of guessed. For anyone building a similar system, the defects found here suggest where to spend effort, which is on the path between a decision and the water and on reporting failures as failures.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The first users are commercial orchid growers in Sri Lanka who run several shade houses. They are small and medium enterprises, a size that dominates the sector according to the Export Development Board [2]. Nurseries and collections that hold *Vanda* and related epiphytes are a second group, and so are growers of the same orchids elsewhere in the humid tropics.

For these users the value is that each section is watered at the right time and for the right length without a probe on any plant, humidity is raised without wetting the roots, and feeds go only with water, at reduced strength and never to a dormant plant. The grower buys only the sensors a house needs, because the calibration decides that, and the spare nodes can be used elsewhere. Every pour is confirmed by the controller that ran it, and the owner can give workers operator or viewer access.

8.2 Market and competitive position

Growers today water by hand or with timers and mist lines, and generic irrigation controllers add a soil moisture threshold [24], which bare root *Vanda* cannot use. Integrated orchid platforms such as OrchiZen [27] advise the grower and leave the action to the grower. This component sits between the two. It acts, confirms what it did, plans per section, and is built for epiphytes from commodity parts. Against established products its weaknesses are a short field record and its dependence on Wi-Fi and the cloud service. Table 8.1 gives the cost basis of the hardware in Sri Lankan rupees (LKR).

Table 8.1: Cost basis of the hardware

Item	Qty per house	Unit cost (LKR)	Basis
Sensor node (ESP32, DHT22, BH1750)	1 per section	2,140	receipts, April 2026, with no tray probe since 1 Oct 2026
NodeMCU ESP32 (Wi-Fi + Bluetooth)		1400	receipt
DHT22		340	receipt
BH1750		400	receipt
4 channel relay module	1	250	receipt
12 V pump (watering, humidity tray)	2	500	receipt
Peristaltic dosing pump (future)	1	1500	receipt
Tubing, trays, power banks, 12 V supply	per house	not costed	

Source: project purchase records

8.3 Business or sustainability model

I see three sources of revenue. The first is the hardware kit, sold per house. The second is a subscription per farm that pays for hosting, alarms, model updates and support and includes the admin account with which the owner manages the farm's users. The third is calibration as a service, in which an installer brings a set of nodes, calibrates the house, leaves only the nodes it needs and reuses the rest. That lowers the upfront cost, which is a main barrier to small producers adopting irrigation technology [23]. Running costs are small. One server at USD 6.60 per month serves every farm as a separate tenant, and Firebase Cloud Messaging has no per message charge.

8.4 Scalability intellectual property and compliance

The design scales by adding tenants and houses without code changes. One server carries the present farm, and its capacity for many farms has not been measured. The firmware, backend and application are the group's own work built on open source

libraries under permissive licences (scikit-learn, PyKrige, PySensors, FastAPI, React Native), whose terms allow commercial use with attribution, and the ERA5 data obtained through Open-Meteo is used under a licence that requires attribution. No personal data beyond users' account details is processed. Before the product is sold, the database rules must be closed, and electrical installation in wet houses should follow local wiring practice.

8.5 Contribution to the Sustainable Development Goals

The component contributes most directly to SDG 6, Clean Water and Sanitation (target 6.4, water-use efficiency), by watering each section for its own conditions instead of a whole house on one routine, and to SDG 12, Responsible Consumption and Production, by feeding at reduced strength, only with water and never to dormant plants. Low cost monitoring for small growers supports SDG 8, Decent Work and Economic Growth, and SDG 9, Industry, Innovation and Infrastructure. These are intended contributions, because water and fertilizer savings have not yet been measured on the farm.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

Vanda orchids have no substrate to measure, and the decision their growers face each day is when and for how long to water, not whether. This thesis presented a system that makes that decision for every section of a shade house from air measurements alone. It keeps the humidity around the plants in band with a tray that the models fill at the right time and the grower sizes, delivers fertilizer only with water, works out how many sensors a house needs from the house's own readings, and carries out its decisions through a controller that confirms every pour. The first hours on the farm show sections that differ by 3.84 °C on average, which supports the premise that a house has to be managed as several climates.

9.2 Achievement of objectives

Table 9.1 sets each objective of Section 1.5 against what was achieved and the evidence for it.

Table 9.1: Achievement of the objectives

Objective	Status	Evidence
1 Sensing node and pump controller	Achieved	Four nodes on the farm, with store-and-forward, hotspot setup and acknowledged pours tested (Section 5.4)
2 Watering time and duration models	Achieved	8.42 min out of time, 8.41 min on unseen weather (Section 6.1.1)
3 Humidity tray loop	Achieved	AUC 0.986 on unseen years, grower set amount (Section 4.4.6)
4 Fertilization schedule	Achieved	Schedule, guards and doser tested(Section 4.4.7)
5 Sensor number and placement	Achieved	Known answers recovered, H1 calibrated (Section 6.1.5)
6 Application, alarms, safety, end-to-end verification	Achieved	19 scenarios passed, 25 of 25 defects fixed (Section 6.2)

9.3 Individual research and product contributions

My research contribution is the reformulation of *Vanda* irrigation as per section scheduling validated on unseen years, the anticipatory tray controller, and a measured method for sensor placement. My product contribution is the working system, from firmware to application, in use on a farm. Table 1.2 lists the work in detail.

The hardest part for me was the IoT side. I had little knowledge of IoT devices when I started, so the early weeks were slow, and integrating the whole system, from the sensor to the pump, the cloud and the application, took far more effort than any single piece. Several faults cost me days because the obvious cause was not the real one. When the first pumps would not run, the relay lights, the acknowledgements from the node and the whole command path were correct, and the fault was the power supply plug. A light sensor that seemed fixed one morning failed again hours later, and only a small diagnostic sketch that scanned the bus showed that the sensor and the wiring were fine and the problem lay in the firmware. A manual two second watering was running for ten seconds because of a minimum I had set in the backend, and I twice blamed other things before I found it. I also had to learn things that were new to me, such as designing a printed circuit board, and the first board that was made did not connect the ESP32 to the sensors, so the nodes on the farm are wired by hand. On the research side, the main difficulty was that no record of growers' watering decisions exists, which is why the targets come from a rule.

9.4 Future directions

The first task is growers' corrections to the plan and the condition of the plants should be recorded over a season, and the watering models retrained on them. A current sensor in the pump supply would let the system notice a pump that is commanded but draws no current. Finally, delivery at each outlet, the nodes' power use and real water savings still have to be measured, and the carrier board remade with its tracks checked against the firmware's pin map.

REFERENCES

[1] D. M. U. B. Dhanasekera, "Cut flower production in Sri Lanka," in *Cut Flower Production in Asia*, RAP Publication 1998/14. Bangkok, Thailand: FAO Regional Office for Asia and the Pacific, 1998.

[2] Sri Lanka Export Development Board, "Industry capability report: Floriculture industry," Colombo, Sri Lanka, 2021. [Online]. Available: https://www.srilankabusiness.com/ebooks/industry-capability-report-floriculture-industry-2021.pdf

[3] G. Zotz and U. Winkler, "Aerial roots of epiphytic orchids: The velamen radicum and its role in water and nutrient uptake," *Oecologia*, vol. 171, no. 3, pp. 733–741, 2013, doi: 10.1007/s00442-012-2575-6.

[4] C. S. Hew and J. W. H. Yong, *The Physiology of Tropical Orchids in Relation to the Industry*, 2nd ed. Singapore: World Scientific, 2004.

[5] New York Botanical Garden, "How do I care for the orchid Ascocenda, Vanda hybrid?," NYBG LibAnswers. [Online]. Available: https://libanswers.nybg.org/faq/223293 (accessed Oct. 10, 2026).

[6] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, "Crop evapotranspiration: Guidelines for computing crop water requirements," FAO Irrigation and Drainage Paper 56, Food and Agriculture Organization of the United Nations, Rome, Italy, 1998.

[7] C. Grossiord, T. N. Buckley, L. A. Cernusak, K. A. Novick, B. Poulter, R. T. W. Siegwolf, J. S. Sperry, and N. G. McDowell, "Plant responses to rising vapor pressure deficit," *New Phytologist*, vol. 226, no. 6, pp. 1550–1566, 2020, doi: 10.1111/nph.16485.

[8] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012, doi: 10.1016/j.ins.2011.12.028.

[11] D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017, doi: 10.1111/ecog.02881.

[12] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn, "Bias in random forest variable importance measures: Illustrations, sources and a solution," *BMC Bioinformatics*, vol. 8, Art. no. 25, 2007, doi: 10.1186/1471-2105-8-25.

[13] H. Hersbach *et al.*, "The ERA5 global reanalysis," *Quarterly Journal of the Royal Meteorological Society*, vol. 146, no. 730, pp. 1999–2049, 2020, doi: 10.1002/qj.3803.

[14] P. Zippenfenig, "Open-Meteo.com Weather API," Zenodo, 2023, doi: 10.5281/zenodo.7970649.

[15] G. Matheron, "Principles of geostatistics," *Economic Geology*, vol. 58, no. 8, pp. 1246–1266, 1963, doi: 10.2113/gsecongeo.58.8.1246.

[16] N. A. C. Cressie, *Statistics for Spatial Data*, rev. ed. New York, NY, USA: Wiley, 1993.

[17] B. Murphy, S. Müller, and R. Yurchak, "GeoStat-Framework/PyKrige," Zenodo, software, version 1.7.2 used, doi: 10.5281/zenodo.3738604.

[18] A. Krause, A. Singh, and C. Guestrin, "Near-optimal sensor placements in Gaussian processes: Theory, efficient algorithms and empirical studies," *Journal of Machine Learning Research*, vol. 9, pp. 235–284, 2008.

[19] K. Manohar, B. W. Brunton, J. N. Kutz, and S. L. Brunton, "Data-driven sparse sensor placement for reconstruction: Demonstrating the benefits of exploiting known patterns," *IEEE Control Systems Magazine*, vol. 38, no. 3, pp. 63–86, 2018, doi: 10.1109/MCS.2018.2810460.

[20] B. M. de Silva, K. Manohar, E. Clark, B. W. Brunton, S. L. Brunton, and J. N. Kutz, "PySensors: A Python package for sparse sensor placement," *Journal of Open Source Software*, vol. 6, no. 58, p. 2828, 2021, doi: 10.21105/joss.02828.

[21] R. P. G. S. Maleesha and P. D. S. Silva, "Analyzing the influence of automated water distribution systems on precision irrigation for orchids: A case study using Dendrobium Phalaenopsis orchid group," in *Proc. 9th Int. Conf. Information Technology Research (ICITR)*, 2024, pp. 1–6, doi: 10.1109/ICITR64794.2024.10857793.

[22] R. A. Fauzy, H. H. Nuha, and A. H. A. Absa, "Implementation of monitoring system and prediction of room humidity for orchid," in *Proc. IEEE Asia Pacific Conf. Wireless and Mobile (APWiMob)*, Bandung, Indonesia, 2022, pp. 1–5, doi: 10.1109/APWiMob56856.2022.10014029.

[23] Z. Drouaillet-Mejía, E. Fernández-Echeverría, W. A. Silva-Martínez, Y. N. Ortega-Gijón, G. Fernández-Lambert, and L. E. García-Santamaría, "Water efficiency in polycultures for small-scale agriculture," *Revista Mexicana de Ciencias Agrícolas*, vol. 16, no. 6, Art. no. e3821, 2025, doi: 10.29312/remexca.v16i6.3821.

[24] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, "IoT-based smart irrigation systems: An overview on the recent trends on sensors and IoT

systems for irrigation in precision agriculture," *Sensors*, vol. 20, no. 4, p. 1042, 2020, doi: 10.3390/s20041042.

[25] M. B. Tephila, R. A. Sri, R. Abinaya, *et al.*, "Automated smart irrigation system using IoT with sensor parameter," in *Proc. Int. Conf. Electronics and Renewable Systems (ICEARS)*, 2022, doi: 10.1109/ICEARS53579.2022.9751993.

[26] A. Goap, D. Sharma, A. K. Shukla, and C. R. Krishna, "An IoT based smart irrigation management system using machine learning and open source technologies," *Computers and Electronics in Agriculture*, vol. 155, pp. 41–49, 2018, doi: 10.1016/j.compag.2018.09.040.

[27] D. Wijendra, V. Jayasinghearachchi, O. A. P. Dilshan, H. M. K. C. B. Herath, Y. M. T. N. S. Yapa, and K. D. M. M. Rathnasiri, "OrchiZen: Hybrid integrated smart farming system for orchid plantations," in *Proc. IEEE Int. Conf. Automatic Control and Intelligent Systems (I2CACIS)*, 2025, pp. 164–169, doi: 10.1109/I2CACIS65476.2025.11101702.

[28] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004, doi: 10.2307/25148625.

[29] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[30] Google, "Firebase Realtime Database," Firebase documentation. [Online]. Available: https://firebase.google.com/docs/database (accessed Oct. 10, 2026).

[31] S. Ramírez, "FastAPI," software documentation. [Online]. Available: https://fastapi.tiangolo.com (accessed Oct. 10, 2026).

[32] Meta Platforms, "React Native," software documentation. [Online]. Available: https://reactnative.dev (accessed Oct. 10, 2026).

[33] Google, "Firebase Cloud Messaging," Firebase documentation. [Online]. Available: https://firebase.google.com/docs/cloud-messaging (accessed Oct. 10, 2026).

[34] Espressif Systems, "ESP32 series datasheet," Shanghai, China. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[35] Aosong Electronics, "Digital-output relative humidity and temperature sensor/module DHT22 (AM2302)," datasheet.

[36] ROHM Semiconductor, "BH1750FVI: Digital 16-bit serial output type ambient light sensor IC," datasheet.

GLOSSARY

Acknowledgement (ack): the record a master controller writes after a pour, stating whether it was done, stopped or refused and how many seconds the relay was on. It is the only evidence the system accepts that water moved.

Anticipatory fill: a humidity tray fill made because the model predicts humidity will fall below the band within three hours, before it has done so.

Calibration: the period after a house is set up during which one node records in each section, so that the placement analysis can decide how many sensors the house keeps.

Co-location: running all nodes side by side for at least 40 minutes before calibration, so each sensor's offset can be measured and removed.

Coverage radius: the distance within which one node's reading stays within the sensor's accuracy of the true value, given as an upper bound when the house varies with direction.

Estimated section: a section without its own node, whose temperature and humidity are estimated by kriging from the measured sections.

Kriging: a geostatistical method estimating a value at an unmeasured location from measured ones, with weights derived from how similarity decays with distance, and giving the variance of each estimate.

Master controller: the node in a house that carries the relay board and runs the pumps, taking pours one at a time from a queue, or from its copy of the day's plan when the cloud is gone.

Offline fallback: watering that a master does on its own, from its copy of the day's plan, when the cloud has been unreachable past a section's planned minute.

Out-of-time validation: testing a model on whole years excluded from its training data.

Quiet node: a node built only to record, which reads every 60 seconds and never polls for commands.

Reanalysis: a reconstruction of past weather made by combining historical observations with a weather model.

Section: a part of a house treated as one microclimate, with its own watering plan.

Tray amount: how many seconds a humidity tray fill pours in a section, set by the grower, or one full tray when not set.

Velamen radicum: the spongy, multi-layered sheath of dead cells covering the aerial roots of epiphytic orchids.

APPENDICES

Appendix A Research instruments or extended results

End-to-end scenarios. Table A.1 lists the scenarios run on October 2026 on the deployed backend, the real application and a simulated house with four virtual quiet boards and a virtual master.

Table A.1: End-to-end scenarios and their results

ID	Scenario	Result
C1	Calibration (create house, link 4 nodes, co-location, analyse, flow)	Pass after F1, injected bias recovered within 0.025 °C
A1	Apply placement and activate the house	Pass, house active
A2	Choose a master controller	Pass, quiet boards no longer offered (F2)
S2	Water Now	Pass, master received 3 s after issue, done in 45 s
S3	Stop a running watering	Pass, stopped, 15 s run
S4	Fill tray	Pass, channel 2, 15 s, six hour rest set
S5	Scheduled watering, automatic care off	Pass, alarm at the planned minute, phone notified within 1 s
S6	Section always automatic, feed due	Pass, engine poured with fertilizer, alarm names 30-10-10
S7	Dry air	Pass, automatic section filled its tray, manual section got an alarm
S13	Repeated alarm	Pass, second notification 5 min 8 s later
H1	Temperature sensor fails	Pass, alarm after 3 min 17 s, opens the Alarm screen
H2	Light sensor fails	Pass, alarm after 11 min 27 s
H3	Node goes silent	Pass, alarm 21 min after the last reading
H4	Wi-Fi outage, store-and-forward	Pass, 18 buffered readings uploaded in time order
H5	Master offline with a queued watering	Pass, alarm after 19 min 20 s, stale command refused, nothing poured
T1	Tray alarm through the full screen path to Fill tray	Pass, tray filled on channel 2 for 15 s

ID	Scenario	Result
F15-live	A section without a sensor is planned	Pass, planned from its estimate and four neighbours
F20-live	Fed watering refused as stale returns the feed	Pass, feed due again, grower told the plants were not fed
A3	Water Now lists the section without a sensor	Pass, shown as estimated, no sensor

Source: Rehearsal/E2E_RESULTS.md

Database layout. All paths are under /tenants/{t}/, where {t} is the farm's tenant identifier, except the shared device registry (Table A.2).

Table A.2: Layout of the Realtime Database used by Component 3

Path	Contents
farm/meta	farm name, time zone, location, automatic care switch
farm/houses/{h}/meta	name, type, dimensions, life cycle (calibrating or active), calibration settings, master MAC, pump channels
…/sections/{s}/meta	name, position in the house, linked node, growth stage
…/sections/{s}/latest	the latest reading with temperature, humidity, light, VPD, fault flag and timestamp
…/sections/{s}/plan	today's plan with time, duration, inputs used, source (measured or estimated) and feed attached
…/sections/{s}/tray	tray decision, amount and who set it, rest period, last fill
…/sections/{s}/fertilizer, forecast, control	feed schedule, today's forecast curve, and mode, pour lengths and routing
farm/history/{h}/{s}/{id}	every reading, keyed by a time ordered identifier
farm/masters/{mac}/queue, running, acks, stop	the master's command queue, pour in progress, acknowledgements, stop request
farm/pendingFeeds, alarms, events, pushTokens	feeds awaiting acknowledgement, alarms, event log and registered phones
devices/{mac}	registry of boards with firmware, build, last seen, assignment and reading interval

REST endpoints. The component's 69 endpoints, grouped by prefix (Table A.3).

Table A.3: REST endpoints of Component 3

Prefix	Endpoints
/api/v2/care	setup, houses, sections, overview, plan-all, tray-check-all, alerts, watering-validation, model-info, farm, location, mode-all, dimensions, lifecycle, master, pumps
/api/v2/care (sections)	plan, tray-check, water, stop, wifi, tray-fill, events, command-status, history, mode, flow, channel, position, durations, name
/api/v2/care/houses	colocation, placement-analysis, analyze-placement, shadehouse-check, health, plan, apply-placement, calibration
/api/v2/devices	list, assign, interval, identify, ping, scan, section lookup
/api/v2/auto	auto-mode, section override, push register and test, alarms and acknowledgement, engine status, run-now
/api/v2/accounts	tenants, users, roles

Hardware. Figure A.1 shows the master's relay board and pumps, and Figure A.2 the first made carrier board, which is not used on the farm.

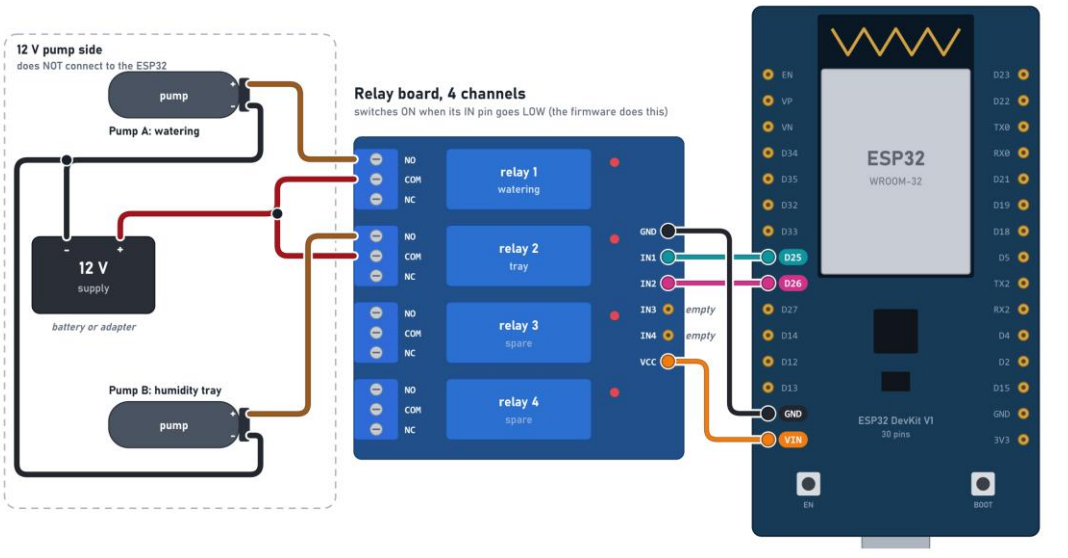
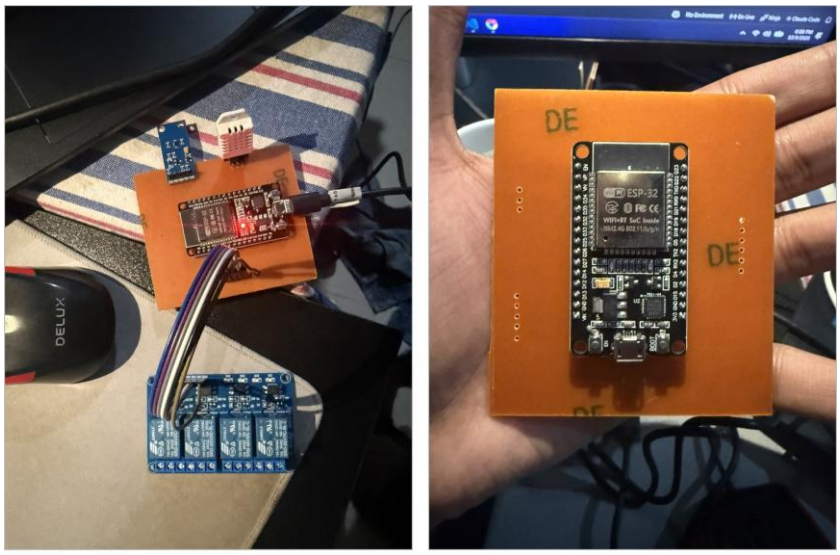


Figure A.1: Master controller: relay board and the two pumps

Source: Orchid sensor node wiring guide, page 4



(a) assembled with relay board (b) ESP32 seated

Figure A.2: The first made carrier board, checked on 9 October 2026

Appendix B Academic integrity and AI use disclosure

Table B.1 lists the generative AI and automated tools used in this work, their purpose, how their output was verified, and the parts of the work they affected.

Table B.1: Use of generative AI and automated tools

Tool	Purpose	Verification performed	Parts affected
Claude Code (Anthropic), from August 2026	Pair programming on the backend, firmware, application and tests, debugging.	Every code change run against the automated tests and, manually reviewed.	Component's code
Gemini AI	Image generation	Manually reviewed	Reports

The design decisions, the farm work, the measurements and the conclusions are the author's, who remains responsible for the accuracy, originality, citations, code, data and analyses in this thesis.